Graph Transformer-Based Financial Representation Learning for Portfolio Optimization with Supervised Return Prediction

Maziar Salahi, MohammadVesal Ahmadian
Department of Mathematics, University of Guilan

Abstract

This paper proposes a graph-based framework for portfolio optimization that integrates financial dependence modeling, Graph Transformer representation learning, contrastive learning, and supervised expected-return prediction. A financial graph is constructed by representing assets as nodes and incorporating multiple pairwise dependence measures as edge attributes. An edge-aware Graph Transformer is then employed to learn relational representations of assets from their historical return information and financial network structure. To improve the robustness of the learned representations, contrastive learning is incorporated through stochastic graph augmentations and a symmetric NT-Xent objective.

A key methodological feature of the proposed framework is the explicit separation between latent representation learning and expected-return estimation. In particular, the latent embedding produced by the Graph Transformer is not directly interpreted as an expected return. Instead, a learnable return decoder maps each asset representation into a predicted next-period return. The resulting predicted-return vector is used as the expected-return input to a Markowitz portfolio optimization model, while the covariance matrix is estimated directly from historical realized returns. This separation provides a financially interpretable connection between representation learning and portfolio optimization and avoids assigning an intrinsic return interpretation to arbitrary latent embedding coordinates.

The framework is evaluated using stock universes from the Dow Jones Industrial Average and the S&P 100 under different training-window lengths and portfolio rebalancing frequencies. Portfolio performance is assessed using return, volatility, drawdown, tail-risk, risk-adjusted performance, concentration, and turnover measures. The proposed formulation provides a unified framework in which the Graph Transformer captures nonlinear cross-asset relationships, the supervised decoder translates learned representations into return forecasts, and Markowitz optimization converts the resulting return and risk estimates into portfolio weights. The revised formulation also establishes a clear basis for evaluating the contribution of graph-based representation learning to portfolio construction without conflating latent features with financial return quantities.

Keywords: Graph Transformer; financial graph; graph neural networks; contrastive learning; return prediction; portfolio optimization; Markowitz model; asset allocation

1 Introduction

Portfolio optimization is a fundamental problem in financial decision making. The classical mean–variance framework introduced by Markowitz [1] formulates portfolio selection as a trade-off between expected return and portfolio risk, where risk is measured through the covariance structure of asset returns. Despite its analytical tractability and continued relevance, the practical performance of mean–variance optimization depends critically on the quality of the estimated expected-return vector and covariance matrix. In particular, estimation error in expected returns can have a substantial effect on the resulting portfolio weights.

Financial asset returns are not independent across assets. Assets may be related through common economic factors, industry membership, market conditions, and other cross-sectional

dependencies. Representing such relationships explicitly can therefore provide additional information for modeling asset returns. Graph-based learning provides a natural framework for this purpose, in which financial assets are represented as nodes and their statistical or economic relationships are represented as edges. Graph neural networks (GNNs) have been widely developed for learning representations from such structured data [2].

Attention mechanisms provide a flexible way of assigning different importance to neighboring nodes. Graph Attention Networks (GATs), for example, learn attention coefficients that determine the relative contribution of neighboring nodes [3]. More recently, Transformer-based graph architectures have incorporated self-attention mechanisms into graph neural networks. In particular, edge-aware Transformer layers allow information contained in edge attributes to participate in the message-passing process. The `TransformerConv` operator implemented in PyTorch Geometric is an example of such an architecture [7, 4].

In financial applications, the graph structure can be constructed from historical relationships among asset returns. At a given portfolio-formation time, each asset can be represented by its recent return history, while the edges between assets can contain measures of statistical dependence. In the present study, the financial graph incorporates several pairwise dependence measures, including Pearson correlation, Spearman correlation, normalized covariance, Kendall correlation, and cosine similarity. The resulting graph provides the Graph Transformer with both asset-specific historical information and cross-sectional information describing relationships among assets.

In addition to graph-based representation learning, contrastive learning has emerged as an effective approach for learning representations from complex data. The central idea is to encourage representations of different stochastic views of the same observation to be similar while distinguishing them from representations associated with other observations [5]. This idea has also been extended to graph data through graph-specific augmentation strategies [6]. In the present framework, contrastive learning is used as an auxiliary objective to encourage the Graph Transformer to learn stable asset representations from perturbed financial graphs.

A central issue, however, arises when a learned latent representation is subsequently introduced into a classical financial model. Let $\boldsymbol{h}_{i,t} \in \mathbb{R}^d$ denote the representation of asset i at time t. Although $\boldsymbol{h}_{i,t}$ may contain information useful for predicting future returns, its individual coordinates do not automatically possess a financial interpretation. In particular, the quantity

$$z_{i,t} = \frac{1}{d} \sum_{k=1}^{d} h_{i,t,k} \tag{1}$$

is simply the arithmetic mean of the latent coordinates. Without an explicit prediction model, there is no theoretical reason for $z_{i,t}$ to represent the expected return of asset i. Consequently, computing a covariance matrix from such latent scalars does not necessarily produce the covariance matrix required by the classical Markowitz model.

To address this issue, this study distinguishes explicitly between a learned latent representation and a financial return estimate. The proposed framework first constructs a financial graph from a rolling window of historical returns. An edge-aware Graph Transformer then maps the graph into an asset-level representation,

$$\boldsymbol{h}_{i,t} = f_\theta(G_t), \tag{2}$$

where G_t denotes the financial graph available at time t. Instead of directly interpreting the latent representation as a return, a trainable decoder maps the representation to a prediction of the next-period asset return:

$$\widehat{r}_{i,t+1} = g_\phi(\boldsymbol{h}_{i,t}). \tag{3}$$

The resulting vector

$$\widehat{\boldsymbol{\mu}}_t = (\widehat{r}_{1,t+1}, \ldots, \widehat{r}_{n,t+1})^\top \tag{4}$$

provides the expected-return input to the portfolio optimization problem. The covariance matrix, in contrast, is estimated directly from historical asset returns. The Graph Transformer therefore improves the estimation of the expected-return vector without changing the financial interpretation of the Markowitz risk term.

The predictive component is combined with the contrastive representation learning objective. The overall training objective is given by

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}}, \tag{5}$$

where $\mathcal{L}_{\text{pred}}$ measures the error in predicting subsequent asset returns and $\mathcal{L}_{\text{CL}}$ is the contrastive loss. Thus, the predictive objective establishes the connection between the learned representation and future asset returns, while the contrastive objective provides an auxiliary representation-learning constraint.

At each portfolio-formation date, the predicted expected-return vector and the historical return covariance matrix are supplied to a constrained Markowitz optimizer:

$$\begin{aligned} \max_{\boldsymbol{w}_t} \quad & \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w}_t - \gamma \boldsymbol{w}_t^\top \boldsymbol{\Sigma}_t \boldsymbol{w}_t \\ \text{s.t.} \quad & \mathbf{1}^\top \boldsymbol{w}_t = 1, \\ & \boldsymbol{w}_t \in \mathcal{W}, \end{aligned} \tag{6}$$

where $\boldsymbol{w}_t$ denotes the portfolio-weight vector, $\gamma > 0$ is the risk-aversion coefficient, and $\mathcal{W}$ represents the investment constraints.

Accordingly, the proposed methodology can be summarized as

$$\boxed{G_t \longrightarrow \boldsymbol{H}_t \longrightarrow \widehat{\boldsymbol{\mu}}_t \longrightarrow \boldsymbol{w}_t^\star,} \tag{7}$$

where

$$\boldsymbol{H}_t = f_\theta(G_t), \qquad \widehat{\boldsymbol{\mu}}_t = g_\phi(\boldsymbol{H}_t), \tag{8}$$

and $\boldsymbol{w}_t^\star$ is obtained from the Markowitz optimization problem.

The main methodological contribution is therefore the integration of cross-sectional financial graph representation learning with expected-return estimation for classical portfolio optimization. The proposed formulation preserves the financial interpretation of the Markowitz model while allowing the expected-return estimator to exploit nonlinear relationships among assets.

The remainder of this paper is organized as follows. Section 2 describes the construction of the financial graph. Section 3 presents the Graph Transformer encoder. Section 4 describes the contrastive representation-learning objective. Section 5 introduces the expected-return decoder. Section 6 formulates the resulting portfolio optimization problem. The training procedure and experimental design are then presented, followed by the empirical results and discussion.

2 Financial Graph Construction

The first stage of the proposed framework is to represent the cross-sectional relationships among financial assets as a sequence of financial graphs. The graph is constructed over a rolling window of historical asset returns. Each asset is represented by a node whose features contain its recent return history, while pairwise statistical dependence measures are used as edge attributes. This construction preserves the terminology and structure of the original model while providing a precise mathematical representation of the information available at each portfolio-formation date.

2.1 Rolling Return Window

Let n denote the number of assets in the investment universe and let T_w denote the length of the rolling historical window. Let $r_{i,\tau}$ denote the return of asset i at trading time τ, where $i = 1, \ldots, n$. For a portfolio-formation date t, the historical return matrix is defined as

$$X_t = \begin{bmatrix} r_{1,t-T_w+1} & r_{1,t-T_w+2} & \cdots & r_{1,t} \\ r_{2,t-T_w+1} & r_{2,t-T_w+2} & \cdots & r_{2,t} \\ \vdots & \vdots & \ddots & \vdots \\ r_{n,t-T_w+1} & r_{n,t-T_w+2} & \cdots & r_{n,t} \end{bmatrix} \in \mathbb{R}^{n \times T_w}. \tag{9}$$

Thus, the i-th row of X_t contains the complete historical return sequence of asset i over the rolling window ending at time t.

Equivalently, the feature vector associated with asset i at time t is

$$\boldsymbol{x}_{i,t} = \begin{bmatrix} r_{i,t-T_w+1} \\ r_{i,t-T_w+2} \\ \vdots \\ r_{i,t} \end{bmatrix} \in \mathbb{R}^{T_w}. \tag{10}$$

In particular,

$$\boldsymbol{x}_{i,t} = \left[r_{i,t-T_w+1}, \ldots, r_{i,t} \right]^\top. \tag{11}$$

The collection of all node features is therefore

$$X_t = \left[\boldsymbol{x}_{1,t}, \boldsymbol{x}_{2,t}, \ldots, \boldsymbol{x}_{n,t} \right]^\top. \tag{12}$$

The rolling-window construction is important for the temporal interpretation of the model. At portfolio-formation time t, all graph information must be constructed exclusively from observations available up to and including t. In particular, the return $r_{i,t+1}$ is not used in the construction of X_t or in the edge attributes of G_t.

2.2 Node Set

The set of financial assets is represented by the node set

$$V = \{v_1, v_2, \ldots, v_n\}, \tag{13}$$

where node v_i corresponds to asset i. Each node v_i is associated with the historical feature vector $\boldsymbol{x}_{i,t}$ defined in Equation (10).

Consequently, at time t, the graph contains n nodes and each node carries a T_w-dimensional feature vector. The node features therefore provide the Graph Transformer with the recent return history of each individual asset.

2.3 Pairwise Dependence Measures

The node features describe the individual historical behavior of the assets. To incorporate cross-sectional relationships, the model additionally assigns edge attributes to pairs of assets.

For each ordered pair of assets (i, j), let

$$\boldsymbol{a}_{ij,t} = \begin{bmatrix} a_{ij,t}^{(1)} \\ a_{ij,t}^{(2)} \\ \vdots \\ a_{ij,t}^{(q)} \end{bmatrix} \in \mathbb{R}^q \tag{14}$$

denote the vector of pairwise dependence measures between assets i and j calculated over the same rolling window used to construct X_t.

In the original implementation, five pairwise dependence measures are used:

$$q = 5, \tag{15}$$

with

$$\boldsymbol{a}_{ij,t} = \begin{bmatrix} \rho_{ij,t}^{\text{Pearson}} \\ \rho_{ij,t}^{\text{Spearman}} \\ c_{ij,t}^{\text{Cov}} \\ \tau_{ij,t}^{\text{Kendall}} \\ s_{ij,t}^{\text{Cosine}} \end{bmatrix}. \tag{16}$$

Thus, the edge attribute vector contains information from Pearson correlation, Spearman correlation, normalized covariance, Kendall correlation, and cosine similarity.

2.3.1 Pearson Correlation

The Pearson correlation between assets i and j over the rolling window is

$$\rho_{ij,t}^{\text{Pearson}} = \frac{\sum_{\tau=t-T_w+1}^{t} \left(r_{i,\tau} - \bar{r}_{i,t}\right)\left(r_{j,\tau} - \bar{r}_{j,t}\right)}{\sqrt{\sum_{\tau=t-T_w+1}^{t} \left(r_{i,\tau} - \bar{r}_{i,t}\right)^2} \sqrt{\sum_{\tau=t-T_w+1}^{t} \left(r_{j,\tau} - \bar{r}_{j,t}\right)^2}}, \tag{17}$$

where

$$\bar{r}_{i,t} = \frac{1}{T_w} \sum_{\tau=t-T_w+1}^{t} r_{i,\tau}. \tag{18}$$

The Pearson correlation captures linear dependence between the return histories of the two assets.

2.3.2 Spearman Correlation

The Spearman correlation is computed from the ranks of the observations in the two return series. Let $\text{rank}(r_{i,\tau})$ denote the rank of $r_{i,\tau}$ within the rolling window. Then

$$\rho_{ij,t}^{\text{Spearman}} = \text{Corr}\left(\text{rank}(r_{i,\tau}), \text{rank}(r_{j,\tau})\right), \tag{19}$$

where the correlation is calculated over $\tau = t - T_w + 1, \ldots, t$.

This measure provides a rank-based dependence measure and therefore captures monotonic relationships that need not be strictly linear.

2.3.3 Normalized Covariance

The third edge attribute is the normalized covariance between the two return series. Denoting the sample covariance by $\widehat{\text{Cov}}_t(r_i, r_j)$, the corresponding normalized quantity can be represented as

$$c_{ij,t}^{\text{Cov}} = \frac{\widehat{\text{Cov}}_t(r_i, r_j)}{s_{i,t} s_{j,t}}, \tag{20}$$

where $s_{i,t}$ and $s_{j,t}$ are the sample standard deviations of the returns of assets i and j over the rolling window.

The resulting quantity is scale-normalized and provides information about the co-movement of the two return series.

2.3.4 Kendall Correlation

The fourth edge attribute is Kendall's rank correlation. It is denoted by

$$\tau_{ij,t}^{\text{Kendall}} = \tau\left(\{r_{i,\tau}\}_{\tau=t-T_w+1}^{t}, \{r_{j,\tau}\}_{\tau=t-T_w+1}^{t}\right). \tag{21}$$

Kendall correlation is based on the concordance and discordance of pairs of observations and provides another rank-based measure of dependence.

2.3.5 Cosine Similarity

The fifth edge attribute is cosine similarity between the two return-history vectors. It is defined as

$$s_{ij,t}^{\text{Cosine}} = \frac{\boldsymbol{x}_{i,t}^{\top}\boldsymbol{x}_{j,t}}{\|\boldsymbol{x}_{i,t}\|_2\|\boldsymbol{x}_{j,t}\|_2}, \tag{22}$$

where $\|\cdot\|_2$ denotes the Euclidean norm.

Unlike covariance-based measures, cosine similarity measures the angular similarity between the two return-history vectors.

2.4 Financial Graph Definition

Using the node set, node features, and pairwise edge attributes defined above, the financial graph at time t is represented by

$$G_t = (V, E_t, X_t, A_t), \tag{23}$$

where

- V is the set of n financial-asset nodes;
- E_t is the edge set;
- $X_t \in \mathbb{R}^{n\times T_w}$ is the node-feature matrix;
- A_t contains the edge attributes $\boldsymbol{a}_{ij,t}$.

The edge set used in the original implementation is

$$E_t = \{(v_i, v_j) \mid i, j = 1, \ldots, n\}. \tag{24}$$

Therefore, the implemented financial graph is a complete directed graph: for every ordered pair (i, j), an edge (v_i, v_j) exists.

In particular,

$$(v_i, v_j) \in E_t \qquad \forall i, j \in \{1, \ldots, n\}, \tag{25}$$

including

$$(v_i, v_i) \in E_t. \tag{26}$$

Thus, self-pairs are included in the implemented graph.

The complete graph construction allows the Graph Transformer to process information from every asset when generating the representation of a given asset. Specifically, the representation learned for asset i can depend not only on its own historical return vector $\boldsymbol{x}_{i,t}$ but also on the historical information and pairwise dependence attributes associated with the other assets.

2.5 Information Available at the Portfolio-Formation Date

The timing of the graph construction is important for avoiding look-ahead bias. At time t, the graph

$$G_t = (V, E_t, X_t, A_t) \tag{27}$$

is constructed exclusively from the observations

$$\{r_{i,\tau} : i = 1, \ldots, n,\ \tau = t - T_w + 1, \ldots, t\}. \tag{28}$$

Consequently,

$$G_t \perp r_{t+1} \quad \text{with respect to the information available at time } t, \tag{29}$$

in the sense that the construction of G_t does not use the future return vector

$$\boldsymbol{r}_{t+1} = (r_{1,t+1}, \ldots, r_{n,t+1})^\top. \tag{30}$$

The future return vector is used only subsequently as the prediction target during model training or as the realized return for evaluating the portfolio formed at time t.

This distinction is essential because the proposed framework is intended for out-of-sample portfolio formation. The graph provides the information set available before the next-period return is realized.

2.6 Summary of the Graph Representation

The complete graph construction can be summarized as

$$\{r_{i,t-T_w+1}, \ldots, r_{i,t}\}_{i=1}^{n} \longrightarrow X_t, \tag{31}$$

together with

$$\{\boldsymbol{a}_{ij,t}\}_{i,j=1}^{n} \longrightarrow A_t, \tag{32}$$

which produces

$$\boxed{G_t = (V, E_t, X_t, A_t).} \tag{33}$$

The resulting graph is then supplied to the Graph Transformer encoder described in Section 3. The encoder maps the node features and edge-level dependence information into asset-level latent representations,

$$\boldsymbol{H}_t = f_\theta(G_t), \qquad \boldsymbol{H}_t \in \mathbb{R}^{n \times d}, \tag{34}$$

which are subsequently used by the proposed expected-return estimation module.

3 Graph Transformer Encoder

The financial graph constructed in Section 2 contains both asset-level historical return information and pairwise dependence information. To extract a representation for each asset while jointly exploiting these two sources of information, a Graph Transformer encoder is employed.

Let

$$G_t = (V, E_t, X_t, A_t) \tag{35}$$

denote the financial graph at time t. The Graph Transformer encoder maps this graph into an asset-level latent representation matrix

$$H_t = f_\theta(G_t), \qquad H_t \in \mathbb{R}^{n \times d}, \tag{36}$$

where θ denotes the trainable parameters of the encoder and d is the dimension of the learned representation. The representation of asset i is the i-th row of H_t:

$$\boldsymbol{h}_{i,t} \in \mathbb{R}^d, \qquad H_t = \begin{bmatrix} \boldsymbol{h}_{1,t}^\top \\ \boldsymbol{h}_{2,t}^\top \\ \vdots \\ \boldsymbol{h}_{n,t}^\top \end{bmatrix}. \tag{37}$$

The representation $\boldsymbol{h}_{i,t}$ is therefore not an observed financial quantity. It is a learned latent vector that summarizes information about asset i and its relationships with the other assets in the graph.

3.1 Edge-Aware Graph Transformer Layer

The encoder follows the edge-aware attention mechanism implemented by the `TransformerConv` operator. Let $\boldsymbol{h}_i^{(\ell)}$ denote the representation of node i at layer ℓ, and let $\boldsymbol{a}_{ij,t}$ denote the edge attributes associated with the directed edge (v_i, v_j).

For an edge (v_i, v_j), the query associated with node i is obtained as

$$\boldsymbol{q}_i^{(\ell)} = W_Q^{(\ell)} \boldsymbol{h}_i^{(\ell)}, \tag{38}$$

while the key associated with node j incorporates both the node representation and the edge attributes:

$$\boldsymbol{k}_{ij}^{(\ell)} = W_K^{(\ell)} \boldsymbol{h}_j^{(\ell)} + W_{E,K}^{(\ell)} \boldsymbol{a}_{ij,t}. \tag{39}$$

Similarly, the value transmitted from node j to node i is constructed as

$$\boldsymbol{v}_{ij}^{(\ell)} = W_V^{(\ell)} \boldsymbol{h}_j^{(\ell)} + W_{E,V}^{(\ell)} \boldsymbol{a}_{ij,t}. \tag{40}$$

The attention score for the directed edge (v_i, v_j) is then calculated using the scaled dot product

$$e_{ij}^{(\ell)} = \frac{\left(\boldsymbol{q}_i^{(\ell)}\right)^\top \boldsymbol{k}_{ij}^{(\ell)}}{\sqrt{d_k}}, \tag{41}$$

where d_k denotes the dimension of the key and query representations.

For each receiving node i, the attention coefficients are normalized over its incoming neighborhood:

$$\alpha_{ij}^{(\ell)} = \frac{\exp\left(e_{ij}^{(\ell)}\right)}{\sum_{k\in\mathcal{N}_t(i)} \exp\left(e_{ik}^{(\ell)}\right)}, \qquad j \in \mathcal{N}_t(i), \tag{42}$$

where $\mathcal{N}_t(i)$ denotes the set of nodes connected to node i in G_t.

Because the graph in the present implementation is complete and includes self-pairs,

$$\mathcal{N}_t(i) = \{1, \ldots, n\}. \tag{43}$$

Consequently, the representation of each asset can incorporate information from all assets in the investment universe.

The aggregated message received by node i is

$$\boldsymbol{m}_i^{(\ell)} = \sum_{j\in\mathcal{N}_t(i)} \alpha_{ij}^{(\ell)} \boldsymbol{v}_{ij}^{(\ell)}. \tag{44}$$

The output of the Graph Transformer layer can consequently be expressed schematically as

$$\widetilde{\boldsymbol{h}}_i^{(\ell+1)} = W_R^{(\ell)} \boldsymbol{h}_i^{(\ell)} + \boldsymbol{m}_i^{(\ell)}, \tag{45}$$

where the first term represents the transformed information associated with the receiving node and the second term represents the attention-weighted information propagated through the graph.

3.2 Multi-Head Attention

The encoder employs multi-head attention. Let M denote the number of attention heads. For head m, the attention mechanism produces

$$\boldsymbol{m}_{i,m}^{(\ell)} = \sum_{j\in\mathcal{N}_t(i)} \alpha_{ij,m}^{(\ell)} \boldsymbol{v}_{ij,m}^{(\ell)}. \tag{46}$$

The outputs of the attention heads are combined to form the representation of node i at the next layer. Depending on the implementation of the Transformer layer, the head outputs are combined through the corresponding multi-head aggregation operation.

The use of multiple attention heads allows the encoder to learn different patterns of cross-asset interaction simultaneously. In the financial setting, different attention heads can therefore assign different importance to relationships among assets based on their node features and edge attributes.

3.3 Encoder Architecture

The Graph Transformer encoder consists of three graph-transformer layers. The first two layers use hidden representations of dimension 64 and 4 attention heads. The final layer produces an asset-level representation of dimension

$$d = 32. \tag{47}$$

Thus, after the final encoder layer,

$$H_t \in \mathbb{R}^{n\times 32}, \tag{48}$$

and

$$\boldsymbol{h}_{i,t} \in \mathbb{R}^{32}. \tag{49}$$

The overall encoder can therefore be represented as the composition

$$f_\theta = f_{\theta_3}^{(3)} \circ f_{\theta_2}^{(2)} \circ f_{\theta_1}^{(1)}, \tag{50}$$

so that

$$H_t = f_\theta(G_t). \tag{51}$$

The architecture uses residual connections, layer normalization, and the Gaussian Error Linear Unit (GELU) activation. For a generic intermediate representation $\boldsymbol{u}$, the GELU activation is

$$\mathrm{GELU}(u) = u\Phi(u), \tag{52}$$

where $\Phi(\cdot)$ is the cumulative distribution function of the standard normal distribution.

The residual structure can be written schematically as

$$\boldsymbol{h}_i^{(\ell+1)} = \mathrm{LN}\left(\boldsymbol{h}_i^{(\ell)} + \mathrm{GELU}\left(\widetilde{\boldsymbol{h}}_i^{(\ell+1)}\right)\right), \tag{53}$$

where $\mathrm{LN}(\cdot)$ denotes layer normalization. The exact placement of normalization and activation follows the implementation of the Graph Transformer architecture.

3.4 Initial Node Representation

The original node features are the rolling return vectors

$$\boldsymbol{x}_{i,t} \in \mathbb{R}^{T_w}. \tag{54}$$

Before the first graph-transformer layer, these features are mapped into the hidden representation space required by the encoder. Denoting this initial mapping by W_X and $\boldsymbol{b}_X$, we can write

$$\boldsymbol{h}_{i,t}^{(0)} = \varphi\left(W_X \boldsymbol{x}_{i,t} + \boldsymbol{b}_X\right), \tag{55}$$

where $\varphi(\cdot)$ denotes the corresponding nonlinear transformation.

The first Graph Transformer layer then combines this asset-specific representation with information propagated from the other nodes through the edge-aware attention mechanism.

3.5 Interpretation of the Learned Representation

The final vector

$$\boldsymbol{h}_{i,t} = f_\theta(G_t)_i \in \mathbb{R}^{32} \tag{56}$$

is an asset-level latent representation conditioned on the complete financial graph at time t.

It is important to distinguish this representation from an expected return. The coordinates of $\boldsymbol{h}_{i,t}$ are latent features learned by the neural network and do not, by themselves, have the interpretation of return, volatility, covariance, or any other directly observed financial quantity.

Accordingly, the proposed framework does not directly use

$$\boldsymbol{h}_{i,t} \tag{57}$$

as the expected-return estimate. Instead, as described in Section 5, the learned representation is supplied to a separate return-prediction decoder,

$$\widehat{r}_{i,t+1} = g_\phi(\boldsymbol{h}_{i,t}), \tag{58}$$

which explicitly maps the latent representation to the scale of future asset returns.

This distinction preserves the role of the Graph Transformer as a representation-learning mechanism while providing a direct statistical connection between the learned representation and the expected-return input required by the Markowitz model.

3.6 Overall Graph-to-Representation Mapping

Combining the financial graph construction and the Graph Transformer encoder, the mapping from historical financial information to asset-level latent representations is

$$(X_t, A_t) \longrightarrow G_t \longrightarrow H_t, \tag{59}$$

where

$$G_t = (V, E_t, X_t, A_t) \tag{60}$$

and

$$H_t = f_\theta(G_t) = \begin{bmatrix} \boldsymbol{h}_{1,t}^\top \\ \vdots \\ \boldsymbol{h}_{n,t}^\top \end{bmatrix} \in \mathbb{R}^{n\times 32}. \tag{61}$$

The resulting matrix H_t is subsequently used by the contrastive representation-learning procedure and, after training, by the expected-return decoder introduced in the next sections.

4 Contrastive Representation Learning

To improve the quality and robustness of the learned asset representations, a contrastive learning objective is incorporated into the proposed framework. The main purpose of this component is to encourage the Graph Transformer encoder to produce similar representations for the same asset under different admissible perturbations of the financial graph, while separating representations associated with different assets.

Unlike the original formulation in which edge dropout and feature masking were disabled in the implementation, the revised framework explicitly employs graph augmentations so that the contrastive objective has two distinct views of the same financial graph. These augmentations are applied only to the information available at time t and therefore do not introduce information from future observations.

4.1 Construction of Contrastive Views

For each financial graph G_t, two augmented views are generated as

$$G_t^{(1)} = \mathcal{T}_1(G_t), \qquad G_t^{(2)} = \mathcal{T}_2(G_t), \tag{62}$$

where $\mathcal{T}_1$ and $\mathcal{T}_2$ denote stochastic graph transformations.

Two types of perturbations are considered: edge dropout and node-feature masking. Let p_e denote the edge-dropout probability and let p_x denote the feature-masking probability. The corresponding transformations can be represented schematically as

$$\mathcal{T}(G_t) = \mathcal{T}_{\text{mask}}\left(\mathcal{T}_{\text{edge}}(G_t)\right). \tag{63}$$

Under edge dropout, a subset of the edges in E_t is randomly removed from the augmented graph. Under feature masking, selected components of the node-feature matrix X_t are masked.

The values of p_e and p_x are treated as hyperparameters and must be specified consistently with the final experimental implementation.

It is important that these perturbations preserve the identity of each asset. Consequently, the asset corresponding to node v_i in the first view remains the same asset represented by node v_i in the second view. Thus, the two views provide different realizations of the same underlying financial information rather than two unrelated observations.

4.2 Shared Graph Transformer Encoder

Both augmented graphs are processed by the same Graph Transformer encoder introduced in Section 3. Parameter sharing is essential because the two views must be mapped into the same representation space. Accordingly,

$$H_t^{(v)} = f_\theta\left(G_t^{(v)}\right), \qquad v \in \{1,2\}, \tag{64}$$

where

$$H_t^{(v)} = \left[h_{1,t}^{(v)}, \ldots, h_{n,t}^{(v)}\right]^\top \in \mathbb{R}^{n\times d}. \tag{65}$$

Here, $h_{i,t}^{(v)} \in \mathbb{R}^d$ denotes the latent representation of asset i obtained from view v. Because the same encoder parameters θ are used for both views, differences between $h_{i,t}^{(1)}$ and $h_{i,t}^{(2)}$ are attributable to the graph perturbations rather than to different encoder parameterizations.

4.3 Projection Head

Following the Graph Transformer encoder, a nonlinear projection head is used for the contrastive objective. For each asset and each view,

$$p_{i,t}^{(v)} = g_\psi\left(h_{i,t}^{(v)}\right), \qquad v \in \{1,2\}, \tag{66}$$

where $g_\psi(\cdot)$ denotes the projection head.

Consistent with the architecture described for the original model, the projection head maps the 32-dimensional encoder representation to a 64-dimensional hidden layer and then back to a 32-dimensional projection:

$$\mathbb{R}^{32} \longrightarrow \mathbb{R}^{64} \longrightarrow \mathbb{R}^{32}. \tag{67}$$

A GELU activation is applied between the two linear transformations. The projection head is used only for the contrastive objective; the projected representation should not be interpreted as an expected return or as a financial return observation.

4.4 Representation Normalization

Before computing the contrastive similarity, each projected representation is normalized to unit Euclidean norm:

$$\widetilde{p}_{i,t}^{(v)} = \frac{p_{i,t}^{(v)}}{\left\|p_{i,t}^{(v)}\right\|_2}, \qquad v \in \{1,2\}. \tag{68}$$

The similarity between two normalized representations is then measured using cosine similarity. Since both representations have unit norm, cosine similarity reduces to their inner product:

$$s\left(\widetilde{p}_{i,t}^{(v)}, \widetilde{p}_{j,t}^{(u)}\right) = \left(\widetilde{p}_{i,t}^{(v)}\right)^\top \widetilde{p}_{j,t}^{(u)}. \tag{69}$$

Thus,

$$-1 \le s\left(\widetilde{p}_{i,t}^{(v)}, \widetilde{p}_{j,t}^{(u)}\right) \le 1. \tag{70}$$

4.5 NT-Xent Contrastive Loss

The contrastive objective is based on the normalized temperature-scaled cross-entropy (NT-Xent) loss. For a given asset i, the representation of the same asset in the other augmented view is considered the positive sample, whereas representations corresponding to other assets are used as negative samples.

Let $\mathcal{B}_t$ denote the set of asset indices participating in the contrastive batch at time t. The loss from view 1 to view 2 for asset i is defined as

$$\ell_{i,t}^{1\to 2} = -\log \frac{\exp\left(s\left(\widetilde{p}_{i,t}^{(1)}, \widetilde{p}_{i,t}^{(2)}\right)/\tau\right)}{\sum_{j\in\mathcal{B}_t} \exp\left(s\left(\widetilde{p}_{i,t}^{(1)}, \widetilde{p}_{j,t}^{(2)}\right)/\tau\right)}, \tag{71}$$

where $\tau > 0$ is the temperature parameter.

The corresponding loss from view 2 to view 1 is

$$\ell_{i,t}^{2\to 1} = -\log \frac{\exp\left(s\left(\widetilde{p}_{i,t}^{(2)}, \widetilde{p}_{i,t}^{(1)}\right)/\tau\right)}{\sum_{j\in\mathcal{B}_t} \exp\left(s\left(\widetilde{p}_{i,t}^{(2)}, \widetilde{p}_{j,t}^{(1)}\right)/\tau\right)}. \tag{72}$$

The symmetric contrastive loss is consequently given by

$$\mathcal{L}_{\mathrm{CL}} = \frac{1}{2|\mathcal{B}_t|} \sum_{i\in\mathcal{B}_t} \left(\ell_{i,t}^{1\to 2} + \ell_{i,t}^{2\to 1}\right). \tag{73}$$

This objective encourages

$$s\left(\widetilde{p}_{i,t}^{(1)}, \widetilde{p}_{i,t}^{(2)}\right) \tag{74}$$

to be large for the same asset while reducing the relative similarity between representations of different assets.

4.6 Role of Contrastive Learning in the Proposed Framework

The contrastive objective is used as an auxiliary representation-learning criterion. It does not replace the predictive objective required for portfolio optimization. In particular, the latent representation

$$h_{i,t} \tag{75}$$

is not itself interpreted as an expected return.

Instead, the representation learned by the Graph Transformer is subsequently passed to a return-prediction decoder, which produces an estimate of the next-period return:

$$\widehat{r}_{i,t+1} = g_\phi\left(h_{i,t}\right). \tag{76}$$

Consequently, the contrastive objective improves the representation used by the predictive component, while the predictive objective provides the direct financial interpretation required by the portfolio-selection problem.

The complete training objective will therefore combine the predictive and contrastive losses:

$$\mathcal{L} = \mathcal{L}_{\mathrm{pred}} + \lambda_{\mathrm{CL}}\mathcal{L}_{\mathrm{CL}}, \tag{77}$$

where $\lambda_{\mathrm{CL}} \geq 0$ controls the contribution of contrastive representation learning.

The projection head g_ψ is used during contrastive training, whereas the asset representation produced by the Graph Transformer is retained for the subsequent return-prediction task. This separation ensures that the representation-learning mechanism and the financial interpretation of the model remain conceptually distinct.

5 Expected Return Estimation from Learned Representations

The Graph Transformer described in Section 3 produces a latent representation for each asset. These representations contain information extracted from the historical return window and the cross-asset relationships encoded in the financial graph. However, a latent representation does not have the direct financial interpretation of a return observation.

Therefore, the revised framework does not interpret the components of the latent representation as returns. In particular, an operation such as

$$z_{i,t} = \frac{1}{d} \sum_{k=1}^{d} H_{t,i,k} \tag{78}$$

cannot, by itself, be interpreted as an expected return. Such an aggregation is only a scalar summary of a latent vector and does not establish a financial mapping between the latent representation and the future return of asset i.

Instead, the proposed framework introduces an explicit return-prediction decoder that maps the learned asset representation to the next-period return.

5.1 Asset-Level Return Decoder

Let

$$h_{i,t} \in \mathbb{R}^d \tag{79}$$

denote the representation of asset i produced by the Graph Transformer using information available through time t. A return decoder $g_\phi(\cdot)$ is then defined by

$$\widehat{r}_{i,t+1} = g_\phi \left(h_{i,t} \right), \tag{80}$$

where ϕ denotes the parameters of the decoder.

The predicted return vector for all n assets is consequently

$$\widehat{\boldsymbol{\mu}}_t = \begin{bmatrix} \widehat{r}_{1,t+1} \\ \widehat{r}_{2,t+1} \\ \vdots \\ \widehat{r}_{n,t+1} \end{bmatrix} \in \mathbb{R}^n. \tag{81}$$

Thus, each component of $\widehat{\boldsymbol{\mu}}_t$ has a direct financial interpretation:

$$\widehat{\mu}_{i,t} = \widehat{r}_{i,t+1}. \tag{82}$$

The notation $\widehat{\boldsymbol{\mu}}_t$ emphasizes that these quantities are model-based estimates of future expected returns rather than latent features.

5.2 Decoder Architecture

The decoder can be implemented as a feed-forward neural network operating independently on the asset-level representations:

$$g_\phi : \mathbb{R}^d \to \mathbb{R}. \tag{83}$$

For example, with $d = 32$, a two-layer multilayer perceptron can be written as

$$u_{i,t} = \sigma \left(W_1 h_{i,t} + b_1 \right), \tag{84}$$

followed by

$$\widehat{r}_{i,t+1} = W_2 u_{i,t} + b_2, \tag{85}$$

where $\sigma(\cdot)$ denotes the selected nonlinear activation function.

The final layer is linear because the target variable is a real-valued return. The decoder parameters are learned from historical observations and are not obtained by averaging or otherwise directly transforming the coordinates of the latent representation.

The exact hidden-layer dimensions and activation functions should be reported according to the final implementation. They should not be inferred from the Graph Transformer architecture itself.

5.3 Predictive Training Objective

The return decoder is trained using realized future returns as prediction targets. For a training set $\mathcal{D}$ containing asset-window pairs, a mean-squared-error objective can be defined as

$$\mathcal{L}_{\text{pred}} = \frac{1}{|\mathcal{D}|} \sum_{(i,t)\in\mathcal{D}} \left(r_{i,t+1} - \widehat{r}_{i,t+1}\right)^2 . \tag{86}$$

Equivalently, other appropriate regression losses may be used if they are explicitly specified and implemented. The essential requirement is that the decoder be trained against an economically interpretable target, namely the realized future return.

The predictive objective establishes the connection between the latent representation and the portfolio-optimization problem:

$$G_t \longrightarrow H_t \longrightarrow \widehat{\boldsymbol{\mu}}_t. \tag{87}$$

This connection is fundamentally different from treating the latent coordinates themselves as returns.

5.4 Joint Representation and Return Learning

The Graph Transformer and return decoder can be trained jointly with the contrastive objective introduced in Section 4. The resulting objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}}\mathcal{L}_{\text{CL}}, \tag{88}$$

where $\lambda_{\text{CL}} \geq 0$ controls the contribution of the contrastive objective.

The predictive component encourages the learned representation to contain information useful for forecasting the next-period return, while the contrastive component encourages representation stability under admissible perturbations of the financial graph.

Consequently, the encoder is optimized according to two complementary requirements:

1. representations of the same asset should remain consistent under different augmented views of the graph; and

2. the representation should contain information that can be mapped to the asset's future return.

The projection head used for the contrastive objective is not required at portfolio-construction time. After representation learning, the asset representation $h_{i,t}$ is passed through the return decoder to obtain $\widehat{r}_{i,t+1}$.

5.5 Expected Return and Covariance: Different Roles

A crucial distinction is made between the estimation of expected returns and the estimation of portfolio risk.

The expected-return vector is obtained from the learned representation:

$$\widehat{\boldsymbol{\mu}}_t = g_\phi\left(H_t\right), \tag{89}$$

where g_ϕ is applied at the asset level.

In contrast, the covariance matrix used by the Markowitz model remains defined in the original return space. Let

$$\boldsymbol{r}_s = \begin{bmatrix} r_{1,s} \\ r_{2,s} \\ \vdots \\ r_{n,s} \end{bmatrix} \in \mathbb{R}^n \tag{90}$$

denote the vector of realized asset returns at time s. Using the historical observations available in the rolling window, the sample mean is

$$\overline{\boldsymbol{r}}_t = \frac{1}{T_w} \sum_{s=t-T_w+1}^{t} \boldsymbol{r}_s, \tag{91}$$

and the sample covariance matrix is

$$\widehat{\boldsymbol{\Sigma}}_t = \frac{1}{T_w - 1} \sum_{s=t-T_w+1}^{t} \left(\boldsymbol{r}_s - \overline{\boldsymbol{r}}_t\right)\left(\boldsymbol{r}_s - \overline{\boldsymbol{r}}_t\right)^\top. \tag{92}$$

Therefore,

$$\boxed{\widehat{\boldsymbol{\mu}}_t \text{ is learned from the Graph Transformer representation,} \qquad \widehat{\boldsymbol{\Sigma}}_t \text{ is estimated from actual returns.}} \tag{93}$$

This distinction preserves the financial meaning of the Markowitz risk term. The covariance matrix describes the joint variation of actual asset returns and is therefore measured in return space rather than in an arbitrary latent representation space.

5.6 Why the Latent Coordinates Are Not Used as Returns

The latent representation

$$h_{i,t} = \left[h_{i,t,1}, \ldots, h_{i,t,d}\right]^\top \tag{94}$$

is learned without an intrinsic requirement that any particular coordinate correspond to return, volatility, or another directly interpretable financial quantity.

Moreover, latent representations are generally non-identifiable up to transformations that preserve the learning objective. For example, a rotation of the latent coordinate system can produce an equivalent representation while changing the numerical value of each individual coordinate. Consequently, a coordinate-wise average such as

$$\frac{1}{d} \sum_{k=1}^{d} h_{i,t,k} \tag{95}$$

does not possess a stable financial interpretation.

The proposed decoder resolves this issue by learning an explicit mapping

$$h_{i,t} \mapsto \widehat{r}_{i,t+1}, \tag{96}$$

whose output is trained against an observed return target.

Thus, the role of the Graph Transformer is to learn an informative representation, whereas the role of the decoder is to translate that representation into an economically interpretable expected-return estimate.

5.7 Information Flow

The complete information flow of the revised framework up to the expected-return stage is

$$\boxed{X_t \longrightarrow G_t \longrightarrow f_\theta(G_t) = H_t \longrightarrow g_\phi(H_t) = \widehat{\boldsymbol{\mu}}_t.} \tag{97}$$

At no stage is a latent coordinate directly identified with a return observation. The only quantities supplied to the expected-return component of the Markowitz problem are the decoder outputs $\widehat{\mu}_{i,t}$.

This formulation retains the central idea of the original representation-based approach—using Graph Transformer representations before portfolio optimization—while providing an explicit and trainable financial interpretation for the expected-return vector.

6 Markowitz Portfolio Optimization

The learned expected-return vector is incorporated into the classical Markowitz mean–variance portfolio optimization framework. The portfolio optimization problem itself is not replaced by a neural-network allocation rule. Instead, the Graph Transformer and return decoder provide the expected-return information required by the classical optimizer.

At each portfolio-construction time t, the two principal inputs to the optimizer are

$$\widehat{\boldsymbol{\mu}}_t = \begin{bmatrix} \widehat{\mu}_{1,t} \\ \vdots \\ \widehat{\mu}_{n,t} \end{bmatrix} \in \mathbb{R}^n \tag{98}$$

and

$$\widehat{\boldsymbol{\Sigma}}_t \in \mathbb{R}^{n \times n}, \tag{99}$$

where $\widehat{\boldsymbol{\mu}}_t$ is obtained from the return decoder and $\widehat{\boldsymbol{\Sigma}}_t$ is estimated from historical asset returns as described in Section 5.

6.1 Portfolio Weights

Let

$$\boldsymbol{w}_t = \begin{bmatrix} w_{1,t} \\ w_{2,t} \\ \vdots \\ w_{n,t} \end{bmatrix} \in \mathbb{R}^n \tag{100}$$

denote the portfolio-weight vector at time t, where $w_{i,t}$ represents the fraction of total capital allocated to asset i.

Under a fully invested long-only portfolio, the weights satisfy

$$\sum_{i=1}^{n} w_{i,t} = 1, \tag{101}$$

and

$$w_{i,t} \geq 0, \qquad i = 1, \ldots, n. \tag{102}$$

Hence,

$$\mathbf{1}^\top \boldsymbol{w}_t = 1, \qquad \boldsymbol{w}_t \geq \mathbf{0}. \tag{103}$$

These constraints imply that the portfolio is fully invested and does not permit short selling.

6.2 Expected Portfolio Return

Given the predicted expected-return vector $\widehat{\boldsymbol{\mu}}_t$, the expected portfolio return is

$$\widehat{\mu}_{p,t} = \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w}_t. \tag{104}$$

Equivalently,

$$\widehat{\mu}_{p,t} = \sum_{i=1}^{n} w_{i,t}\widehat{\mu}_{i,t}. \tag{105}$$

Thus, the contribution of the Graph Transformer to the Markowitz problem enters through the asset-level estimates $\widehat{\mu}_{i,t}$ produced by the return decoder.

6.3 Portfolio Risk

Portfolio risk is measured using the covariance matrix of the realized asset returns:

$$\widehat{\sigma}_{p,t}^2 = \boldsymbol{w}_t^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w}_t. \tag{106}$$

The corresponding portfolio volatility is

$$\widehat{\sigma}_{p,t} = \sqrt{\boldsymbol{w}_t^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w}_t}. \tag{107}$$

Importantly, $\widehat{\boldsymbol{\Sigma}}_t$ is not computed from the latent representations H_t. It is estimated from the historical return observations. Therefore, the quadratic risk term retains its standard financial interpretation.

6.4 Mean–Variance Optimization

Following the classical mean–variance formulation, the portfolio is obtained by maximizing expected return penalized by portfolio variance:

$$\boxed{\boldsymbol{w}_t^\star = \arg\max_{\boldsymbol{w}_t} \left\{ \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w}_t - \frac{\gamma}{2} \boldsymbol{w}_t^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w}_t \right\}} \tag{108}$$

subject to

$$\mathbf{1}^\top \boldsymbol{w}_t = 1, \qquad \boldsymbol{w}_t \geq \mathbf{0}, \tag{109}$$

where $\gamma > 0$ is the risk-aversion coefficient.

Equivalently, the problem can be written as

$$\boldsymbol{w}_t^\star = \arg\min_{\boldsymbol{w}_t} \left\{ \frac{\gamma}{2} \boldsymbol{w}_t^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w}_t - \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w}_t \right\} \tag{110}$$

subject to the same portfolio constraints.

The two formulations are equivalent because they differ only by multiplication of the objective by -1.

6.5 Risk-Aversion Parameter

The coefficient γ determines the relative importance assigned to expected return and portfolio risk. A larger value of γ places greater weight on the variance penalty, whereas a smaller value places relatively greater emphasis on the predicted expected return.

The value of γ must be specified as part of the experimental design. In the implementation described in the source manuscript, the risk-aversion coefficient is fixed at

$$\gamma = 1. \tag{111}$$

This value should therefore be retained if the revised numerical implementation follows the existing experimental configuration.

6.6 Rolling Portfolio Construction

The optimization is performed repeatedly over rolling information windows. At each decision time t, the available historical returns are used to construct the financial graph:

$$G_t = \mathcal{G}\left(\boldsymbol{r}_{t-T_w+1}, \ldots, \boldsymbol{r}_t\right). \tag{112}$$

The Graph Transformer produces the asset representations

$$H_t = f_\theta(G_t), \tag{113}$$

and the return decoder produces

$$\widehat{\boldsymbol{\mu}}_t = g_\phi(H_t). \tag{114}$$

The covariance matrix is estimated from the same historical information:

$$\widehat{\boldsymbol{\Sigma}}_t = \mathrm{Cov}\left(\boldsymbol{r}_{t-T_w+1}, \ldots, \boldsymbol{r}_t\right). \tag{115}$$

The portfolio weights are then obtained by solving

$$\boldsymbol{w}_t^\star = \underset{\boldsymbol{w}_t \geq \mathbf{0},\, \mathbf{1}^\top \boldsymbol{w}_t = 1}{\arg\max} \left\{ \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w}_t - \frac{\gamma}{2} \boldsymbol{w}_t^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w}_t \right\}. \tag{116}$$

After the portfolio weights have been determined, the subsequent realized return is used only for out-of-sample evaluation and for constructing the next rolling information set. It must not be used in the estimation of $\widehat{\boldsymbol{\mu}}_t$ or $\widehat{\boldsymbol{\Sigma}}_t$ at decision time t.

6.7 Realized Portfolio Return

Once the optimal weights have been obtained, the realized portfolio return in the subsequent period is

$$r_{p,t+1} = \sum_{i=1}^{n} w_{i,t}^\star r_{i,t+1}, \tag{117}$$

or, in vector notation,

$$r_{p,t+1} = \left(\boldsymbol{w}_t^\star\right)^\top \boldsymbol{r}_{t+1}. \tag{118}$$

The realized return is distinct from the predicted expected return:

$$\widehat{\mu}_{p,t} = \left(\boldsymbol{w}_t^\star\right)^\top \widehat{\boldsymbol{\mu}}_t, \tag{119}$$

whereas

$$r_{p,t+1} = \left(\boldsymbol{w}_t^\star\right)^\top \boldsymbol{r}_{t+1}. \tag{120}$$

The first quantity is available at the portfolio-construction stage, while the second becomes available only after the subsequent return is observed.

6.8 Overall Architecture

Combining the preceding components, the revised framework can be summarized as

$$\boxed{\begin{aligned} \text{Historical returns} &\longrightarrow G_t \\ &\longrightarrow H_t = f_\theta(G_t) \\ &\longrightarrow \widehat{\boldsymbol{\mu}}_t = g_\phi(H_t) \\ &\longrightarrow \left(\widehat{\boldsymbol{\mu}}_t, \widehat{\boldsymbol{\Sigma}}_t\right) \\ &\longrightarrow \boldsymbol{w}_t^\star. \end{aligned}} \tag{121}$$

The Graph Transformer therefore modifies the information supplied to the Markowitz optimizer rather than modifying the optimizer itself. The expected-return input is learned from the graph representation, while the risk term remains based on the covariance of actual asset returns.

This formulation also provides a clear separation between the three principal objects in the model:

$$\begin{array}{lcl} H_t & : & \text{latent asset representation,} \\ \widehat{\boldsymbol{\mu}}_t & : & \text{predicted expected-return vector,} \\ \widehat{\boldsymbol{\Sigma}}_t & : & \text{historical return covariance matrix.} \end{array} \tag{122}$$

This separation is essential for maintaining the mathematical interpretation of the mean–variance optimization problem while allowing the Graph Transformer to provide a richer, relational representation of the available financial information.

7 Training Procedure and Rolling-Window Portfolio Construction

The proposed framework is trained and evaluated using a rolling-window procedure. For each outer training window, the financial graphs are constructed exclusively from the historical observations available within that window. The Graph Transformer encoder and the projection head are then trained jointly, after which the projection head is discarded and the trained encoder is used to obtain asset-level representations.

In the revised framework, the return-prediction decoder is trained jointly with the encoder so that the learned representation has a direct connection to the subsequent return-prediction task. The resulting predicted expected-return vector is then supplied to the Markowitz optimizer together with the covariance matrix estimated from realized historical returns.

7.1 Outer Rolling Window

Let T_w denote the length of the historical return window. For a decision time t, the information available to the model is restricted to

$$\mathcal{I}_t = \{\boldsymbol{r}_{t-T_w+1}, \ldots, \boldsymbol{r}_t\}\,. \tag{123}$$

The corresponding financial graph is constructed as

$$G_t = \mathcal{G}\left(\mathcal{I}_t\right). \tag{124}$$

No observation occurring after time t is used in the construction of G_t, its node features, its edge attributes, or its covariance matrix.

For each outer training window, the model parameters are initialized independently. In particular, the original implementation specifies that a new Graph Transformer encoder and projection head are initialized for each outer training window rather than transferring parameters from a previous temporal segment.

This procedure can be represented as

$$(\theta_t, \psi_t, \phi_t) \leftarrow \text{Initialize}(), \tag{125}$$

where θ_t, ψ_t, and ϕ_t denote the encoder, projection-head, and return-decoder parameters, respectively.

7.2 Construction of Training Views

For each training graph G_t, two augmented views are constructed:

$$G_t^{(1)} = \mathcal{T}_1(G_t), \qquad G_t^{(2)} = \mathcal{T}_2(G_t). \tag{126}$$

The two views are passed through the same encoder:

$$H_t^{(1)} = f_{\theta_t}\left(G_t^{(1)}\right), \qquad H_t^{(2)} = f_{\theta_t}\left(G_t^{(2)}\right). \tag{127}$$

For the contrastive component, the corresponding representations are passed through the shared projection head:

$$P_t^{(v)} = g_{\psi_t}\left(H_t^{(v)}\right), \qquad v \in \{1, 2\}. \tag{128}$$

The NT-Xent loss is then evaluated using the two projected views according to Section 4.

7.3 Return-Prediction Branch

In addition to the contrastive branch, the representation of the original financial graph is passed to the return decoder:

$$H_t = f_{\theta_t}(G_t), \tag{129}$$

followed by

$$\widehat{\boldsymbol{\mu}}_t = g_{\phi_t}(H_t). \tag{130}$$

The prediction loss is calculated using the realized next-period returns:

$$\mathcal{L}_{\text{pred},t} = \frac{1}{n}\sum_{i=1}^{n}\left(r_{i,t+1} - \widehat{r}_{i,t+1}\right)^2. \tag{131}$$

The total loss for the training sample is

$$\mathcal{L}_t = \mathcal{L}_{\text{pred},t} + \lambda_{\text{CL}}\mathcal{L}_{\text{CL},t}. \tag{132}$$

Thus, the encoder is simultaneously encouraged to learn stable representations under graph perturbations and representations that contain information relevant to future asset returns.

7.4 Parameter Optimization

The trainable parameters are optimized jointly:

$$\Theta_t = \{\theta_t, \psi_t, \phi_t\}. \tag{133}$$

The optimization problem is

$$\Theta_t^\star = \arg\min_{\Theta_t} \mathcal{L}_{\text{total}}. \tag{134}$$

The original implementation uses the Adam optimizer for jointly training the encoder and projection head. In the revised formulation, the return decoder is included in the same joint optimization so that the representation is explicitly coupled to the return-prediction task.

The learning rate, number of optimization epochs, batch construction, temperature τ, and contrastive-loss weight λ_{CL} should be reported from the final implementation rather than being inferred from the mathematical formulation.

7.5 Discarding the Projection Head

After the training stage is completed, the projection head is discarded:

$$g_{\psi_t} \quad \longrightarrow \quad \text{discarded.} \tag{135}$$

The trained encoder is retained:

$$f_{\theta_t^\star}. \tag{136}$$

This follows the representation-learning structure of the original framework, in which the projection head is used during contrastive training and only the encoder representation is retained after training.

The return decoder, however, is retained because its output is required to produce the expected-return vector used by the Markowitz optimizer.

7.6 Portfolio Construction at the Decision Time

Once the encoder and return decoder have been trained, the representation of the current decision-time graph is obtained:

$$H_t^\star = f_{\theta_t^\star}(G_t). \tag{137}$$

The expected-return vector is then calculated as

$$\widehat{\boldsymbol{\mu}}_t = g_{\phi_t^\star}\left(H_t^\star\right). \tag{138}$$

Independently, the historical covariance matrix is estimated from the realized returns contained in the available information set:

$$\widehat{\boldsymbol{\Sigma}}_t = \mathrm{Cov}\left(\boldsymbol{r}_{t-T_w+1}, \ldots, \boldsymbol{r}_t\right). \tag{139}$$

The portfolio weights are obtained by solving

$$\boldsymbol{w}_t^\star = \underset{\mathbf{1}^\top \boldsymbol{w}=1,\, \boldsymbol{w}\geq \mathbf{0}}{\arg\max} \left\{ \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w} - \frac{\gamma}{2} \boldsymbol{w}^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w} \right\}. \tag{140}$$

The resulting $\boldsymbol{w}_t^\star$ is the portfolio implemented for the subsequent period.

7.7 Out-of-Sample Evaluation

After the portfolio weights have been determined, the next-period return is realized:

$$r_{p,t+1} = \left(\boldsymbol{w}_t^\star\right)^\top \boldsymbol{r}_{t+1}. \tag{141}$$

This observation is not used retroactively in the construction of the portfolio at time t. It becomes part of the information available for a subsequent rolling-window decision.

Accordingly, the temporal ordering is

$$\boxed{\begin{array}{c}\text{historical returns through } t\\ \downarrow\\ G_t\\ \downarrow\\ \text{Graph Transformer training}\\ \downarrow\\ \widehat{\boldsymbol{\mu}}_t,\ \widehat{\boldsymbol{\Sigma}}_t\\ \downarrow\\ \boldsymbol{w}_t^\star\\ \downarrow\\ \boldsymbol{r}_{t+1}\\ \downarrow\\ r_{p,t+1}.\end{array}} \tag{142}$$

This ordering is essential for preventing look-ahead bias.

7.8 Complete Rolling-Window Algorithm

The complete procedure can be summarized as follows.

1. Select the historical information window ending at time t.
2. Construct the financial graph G_t from the historical asset returns and pairwise dependence measures.
3. Initialize the Graph Transformer encoder, contrastive projection head, and return decoder for the outer training window.
4. Construct two augmented views of each training graph.
5. Pass both views through the shared Graph Transformer encoder and projection head.
6. Compute the contrastive loss $\mathcal{L}_{\mathrm{CL}}$.
7. Pass the encoder representation through the return decoder and compute the predictive loss $\mathcal{L}_{\mathrm{pred}}$.
8. Minimize
$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{pred}} + \lambda_{\mathrm{CL}}\mathcal{L}_{\mathrm{CL}} \tag{143}$$
using Adam.
9. After training, discard the projection head and retain the trained encoder and return decoder.
10. Generate the expected-return vector $\widehat{\boldsymbol{\mu}}_t$ for the current decision-time graph.
11. Estimate $\widehat{\boldsymbol{\Sigma}}_t$ from the historical realized returns.
12. Solve the constrained Markowitz optimization problem to obtain $\boldsymbol{w}_t^\star$.
13. Apply $\boldsymbol{w}_t^\star$ to the subsequent period and record the realized portfolio return.

14. Move the rolling window forward and repeat the procedure.

The resulting framework can therefore be represented compactly as

$$\boxed{\begin{aligned}&\text{Returns} \to \text{Financial Graph} \to \text{Graph Transformer}\\ &\qquad \to \text{Asset Representations} \to \text{Return Decoder} \to \widehat{\boldsymbol{\mu}}\\ &\qquad \to \text{Markowitz Optimizer} \to \boldsymbol{w}^{\star} \to \text{Out-of-Sample Return.}\end{aligned}} \tag{144}$$

The contrastive-learning branch operates as an auxiliary representation-learning mechanism within this pipeline. The final portfolio is determined by the classical Markowitz optimization problem using the learned expected-return estimates and the covariance matrix of actual historical returns.

8 Experimental Setup and Parameter Definition

The proposed framework is evaluated using historical daily return data for constituent assets of the Dow Jones Industrial Average (DJIA) and the S&P 100. The implementation is developed in Python using PyTorch and PyTorch Geometric. The Graph Transformer component is implemented using the TransformerConv operator, while the trainable parameters are optimized using Adam.

The experimental design follows a chronological rolling-window structure. Historical observations are divided into training and out-of-sample testing periods. The training observations are used for financial graph construction, representation learning, expected-return estimation, and portfolio optimization, whereas subsequent observations are reserved for out-of-sample evaluation.

8.1 Financial Data

The experiments use two equity universes:

1. the constituents of the Dow Jones Industrial Average (DJIA); and
2. the available constituents of the S&P 100.

The historical market data are obtained using the `yfinance` Python package. Adjusted closing prices are converted into daily simple returns according to

$$r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}, \tag{145}$$

where $P_{i,t}$ denotes the adjusted closing price of asset i at time t.

The resulting return observations constitute the basic input to both the financial graph construction and the subsequent portfolio optimization procedure.

8.2 Chronological Training and Testing

The observations are divided chronologically rather than randomly. This is necessary because the portfolio construction problem is inherently temporal and future observations must not be available when the portfolio decision is made.

For the one-year configuration, the source implementation uses a training period covering 2020–2021 and an out-of-sample testing period extending from 2021 to 2026. The S&P 100 experiment contains 98 available assets, whereas the DJIA experiment contains 30 assets. For the extended-lookback experiments, the outer training-window lengths are 504 and 756 trading days.

The corresponding DJIA experiments use the same out-of-sample testing horizon beginning in 2021.

Thus, the principal outer training-window configurations are

$$T_{\text{outer}} \in \{252, 504, 756\}, \tag{146}$$

corresponding to approximately one-, two-, and three-year historical windows.

8.3 Graph-Construction Parameters

Each financial graph is constructed from an inner rolling return window of length

$$T_w = 20. \tag{147}$$

Consequently,

$$X_t \in \mathbb{R}^{n \times 20}. \tag{148}$$

Each ordered pair of assets, including self-pairs, is connected by an edge. Five pairwise dependence measures are used as edge attributes:

$$q = 5, \tag{149}$$

namely Pearson correlation, Spearman correlation, normalized covariance, Kendall correlation, and cosine similarity.

Accordingly,

$$\boldsymbol{a}_{ij,t} = \begin{bmatrix} \rho_{ij,t}^{\text{Pearson}} \\ \rho_{ij,t}^{\text{Spearman}} \\ c_{ij,t}^{\text{Cov}} \\ \tau_{ij,t}^{\text{Kendall}} \\ s_{ij,t}^{\text{Cosine}} \end{bmatrix} \in \mathbb{R}^5. \tag{150}$$

The edge attributes are calculated from the same historical window used to construct the corresponding node features. Hence, no future return observation is used in the construction of the graph at decision time t.

8.4 Graph Transformer Architecture

The encoder consists of three TransformerConv layers. The first two layers use 64 hidden features and four attention heads, while the final layer produces a 32-dimensional representation for each asset:

$$d = 32. \tag{151}$$

The edge-feature dimension is five, and the Transformer dropout rate is

$$p_{\text{drop}} = 0.1. \tag{152}$$

Layer normalization, GELU activation, and the implemented residual connections are used in the intermediate layers.

The resulting representation is therefore

$$H_t = f_\theta(G_t) \in \mathbb{R}^{n \times 32}, \tag{153}$$

with

$$\boldsymbol{h}_{i,t} \in \mathbb{R}^{32} \tag{154}$$

denoting the learned representation of asset i.

Importantly, $\boldsymbol{h}_{i,t}$ is not interpreted as an expected return. In the revised framework, it is an intermediate latent representation that is mapped to the predicted next-period return by the return decoder.

8.5 Contrastive-Learning Parameters

The projection head used for contrastive learning has the architecture

$$32 \longrightarrow 64 \longrightarrow 32, \tag{155}$$

with GELU as the intermediate activation. The NT-Xent temperature is

$$\tau = 0.2. \tag{156}$$

The source implementation specifies two graph views with different edge-dropout and feature-masking probabilities:

$$p_e^{(1)} = 0.15, \qquad p_x^{(1)} = 0.10, \tag{157}$$

and

$$p_e^{(2)} = 0.20, \qquad p_x^{(2)} = 0.15. \tag{158}$$

These parameters define the stochastic graph perturbations used to generate the two contrastive views.

The contrastive projection head is used only during representation learning. After training, it is discarded, while the Graph Transformer encoder is retained for generating the asset representations.

8.6 Optimization Parameters

The trainable model parameters are optimized using Adam with learning rate

$$\eta = 10^{-3} \tag{159}$$

and weight decay

$$\lambda_{\text{wd}} = 10^{-5}. \tag{160}$$

The source implementation uses

$$N_{\text{epoch}} = 10 \tag{161}$$

training epochs for each outer training window.

For the revised framework, the return decoder parameters are included in the trainable parameter set. Thus,

$$\Theta = \{\theta, \psi, \phi\}, \tag{162}$$

where θ denotes the Graph Transformer parameters, ψ denotes the contrastive projection-head parameters, and ϕ denotes the return-decoder parameters.

The joint objective is

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}}. \tag{163}$$

The exact value of λ_{CL} should be taken from the final revised implementation if it differs from the original code. It should not be inferred from the source manuscript.

8.7 Risk-Aversion Parameter

The Markowitz risk-aversion coefficient is fixed at

$$\gamma = 1. \tag{164}$$

The portfolio is subject to the long-only and fully invested constraints

$$\mathbf{1}^\top \boldsymbol{w} = 1, \qquad \boldsymbol{w} \geq \mathbf{0}. \tag{165}$$

The revised portfolio optimization therefore uses

$$\boldsymbol{w}_t^\star = \underset{\mathbf{1}^\top \boldsymbol{w} = 1,\, \boldsymbol{w} \geq \mathbf{0}}{\arg\max} \left\{ \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w} - \frac{1}{2} \boldsymbol{w}^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w} \right\}. \tag{166}$$

8.8 Rebalancing Frequencies

Two portfolio implementation schemes are considered.

Buy and Hold. The portfolio weights obtained at the beginning of the out-of-sample period are maintained throughout the corresponding testing horizon without portfolio reoptimization.

Periodic Rebalancing. At each rebalancing point, the most recent historical observations are used to reconstruct the financial graphs, generate the asset representations and expected-return estimates, estimate the covariance matrix, and solve the Markowitz problem again.

The rebalancing interval is

$$R \in \{21, 63, 126, 252\}, \tag{167}$$

corresponding approximately to monthly, quarterly, semi-annual, and annual rebalancing, respectively.

8.9 Benchmark Strategies

The source study considers four portfolio references:

1. **MK(Embedding):** the proposed representation-based Markowitz strategy;
2. **MK:** conventional Markowitz optimization based on historical returns;
3. **EW:** equal-weight allocation, with
$$w_i = \frac{1}{n}; \tag{168}$$
4. **S&P 500:** an external market benchmark.

The S&P 500 benchmark is not used as a training or representation-learning dataset. The source implementation trains the proposed model using the DJIA and S&P 100 constituent data.

In the revised methodology, the label "MK(Embedding)" should be used carefully. Since the expected-return vector is no longer obtained by directly treating the latent embedding as a financial return, a more precise name for the proposed method is

$$\text{MK(Decoder)} \tag{169}$$

or

$$\text{Graph Transformer–Markowitz.} \tag{170}$$

If continuity with the original tables and discussion is important, "MK(Embedding)" may be retained as a descriptive label, but the manuscript must explicitly define it as the Markowitz strategy whose expected returns are estimated from Graph Transformer representations through the learned return decoder.

8.10 Summary of Experimental Parameters

The principal parameters of the revised framework are summarized in Table 1.

The principal methodological change relative to the original representation-based formulation is that the 32-dimensional Graph Transformer output is no longer treated as a direct financial quantity. Instead, it serves as the input to a learned return decoder:

$$\boxed{\boldsymbol{h}_{i,t} \longrightarrow \widehat{r}_{i,t+1} \longrightarrow \widehat{\boldsymbol{\mu}}_t \longrightarrow \text{Markowitz.}} \tag{171}$$

Table 1: Main experimental parameters of the revised framework.

Parameter	Value
Inner graph-window length T_w	20 trading days
Outer training windows	252, 504, 756 trading days
Number of edge features q	5
Graph type	Complete directed graph with self-pairs
Transformer layers	3
Hidden dimension	64
Attention heads	4
Final embedding dimension	32
Transformer dropout	0.1
Projection head	$32 \to 64 \to 32$
GELU	Yes
NT-Xent temperature τ	0.2
Learning rate	10^{-3}
Weight decay	10^{-5}
Training epochs per outer window	10
Risk-aversion coefficient γ	1
Rebalancing intervals	21, 63, 126, 252 days
Portfolio constraints	$\mathbf{1}^\top \boldsymbol{w} = 1,\ \boldsymbol{w} \geq 0$

At the same time, the covariance matrix remains defined in the space of realized asset returns:

$$\boxed{\widehat{\boldsymbol{\Sigma}}_t = \mathrm{Cov}\left(\boldsymbol{r}_{t-T_w+1}, \ldots, \boldsymbol{r}_t\right).} \tag{172}$$

This separation ensures that the two statistical inputs to the Markowitz problem have well-defined financial interpretations: $\widehat{\boldsymbol{\mu}}_t$ represents predicted future returns, whereas $\widehat{\boldsymbol{\Sigma}}_t$ represents the covariance structure of realized historical returns.

9 Evaluation Metrics

The proposed portfolio strategies are evaluated using realized out-of-sample portfolio returns. The evaluation considers three groups of measures: risk metrics, performance metrics, and portfolio-composition metrics. This separation allows the analysis to distinguish the realized risk of a strategy, its risk-adjusted performance, and the structural characteristics of the resulting portfolio.

Let

$$r_{p,k} = \sum_{i=1}^{n} w_{i,k}^{\star} r_{i,k} = \left(\boldsymbol{w}_k^{\star}\right)^\top \boldsymbol{r}_k, \tag{173}$$

denote the realized portfolio return at out-of-sample period k, where $\boldsymbol{w}_k^{\star}$ is the portfolio weight vector determined before observing $\boldsymbol{r}_k$.

Let T_{out} denote the number of out-of-sample observations. The mean realized portfolio return is

$$\widehat{\mu}_p = \frac{1}{T_{\text{out}}} \sum_{k=1}^{T_{\text{out}}} r_{p,k}. \tag{174}$$

9.1 Risk Metrics

9.1.1 Portfolio Volatility

Portfolio volatility is measured by the sample standard deviation of the realized out-of-sample portfolio returns:

$$\widehat{\sigma}_p = \sqrt{\frac{1}{T_{\text{out}} - 1} \sum_{k=1}^{T_{\text{out}}} (r_{p,k} - \widehat{\mu}_p)^2}. \tag{175}$$

A lower value of $\widehat{\sigma}_p$ indicates lower dispersion of realized portfolio returns.

9.1.2 Maximum Drawdown

Maximum drawdown measures the largest peak-to-trough decline in the cumulative portfolio value over the evaluation period.

Define the cumulative wealth process by

$$W_k = W_0 \prod_{j=1}^{k} (1 + r_{p,j}), \qquad k = 1, \ldots, T_{\text{out}}, \tag{176}$$

where W_0 is the initial portfolio value.

The running peak is

$$M_k = \max_{1 \le j \le k} W_j. \tag{177}$$

The drawdown at period k is

$$D_k = \frac{W_k - M_k}{M_k}. \tag{178}$$

The maximum drawdown is then

$$\text{MDD} = \min_{1 \le k \le T_{\text{out}}} D_k. \tag{179}$$

Thus, maximum drawdown is reported as a non-positive quantity when the drawdown convention of the source study is followed.

9.1.3 Value at Risk

Value at Risk (VaR) summarizes a loss threshold at a specified confidence level. For a confidence level α, the return-based VaR can be expressed through the lower-tail quantile of the realized portfolio-return distribution:

$$\text{VaR}_\alpha = Q_{1-\alpha}\left(r_{p,1}, \ldots, r_{p,T_{\text{out}}}\right), \tag{180}$$

where $Q_{1-\alpha}$ denotes the corresponding lower-tail empirical quantile under the return convention.

The source study reports

$$\text{VaR}_{0.95}. \tag{181}$$

Because the manuscript reports VaR as a return/loss quantity, negative values indicate losses in the lower tail.

9.1.4 Conditional Value at Risk

Conditional Value at Risk (CVaR) evaluates the expected loss beyond the VaR threshold. Under the return convention used in the reported results, it is defined as the conditional mean of the portfolio returns in the tail beyond VaR_α:

$$\mathrm{CVaR}_\alpha = \mathbb{E}\left[r_p \mid r_p \le \mathrm{VaR}_\alpha\right]. \tag{182}$$

The reported experiments use

$$\mathrm{CVaR}_{0.95}. \tag{183}$$

Both VaR and CVaR are evaluated exclusively on realized out-of-sample portfolio returns.

9.2 Performance Metrics

9.2.1 Sharpe Ratio

With a zero risk-free rate, the Sharpe ratio is calculated as

$$\mathrm{Sharpe} = \frac{\widehat{\mu}_p}{\widehat{\sigma}_p}. \tag{184}$$

This definition follows the formulation used in the source study.

9.2.2 Sortino Ratio

The Sortino ratio replaces total volatility with downside deviation. Let τ denote the target return. The downside deviation can be written as

$$\sigma_d = \sqrt{\frac{1}{T_{\mathrm{out}}} \sum_{k=1}^{T_{\mathrm{out}}} \min\left(r_{p,k} - \tau, 0\right)^2}. \tag{185}$$

The corresponding Sortino ratio is

$$\mathrm{Sortino} = \frac{\widehat{\mu}_p - \tau}{\sigma_d}. \tag{186}$$

The source manuscript describes Sortino as using downside deviation rather than total volatility but does not explicitly specify the target-return convention. Therefore, the value of τ used in the final implementation should be stated explicitly when the numerical results are reported.

9.2.3 Omega Ratio

The Omega ratio compares probability-weighted gains and losses relative to a specified threshold τ. For the empirical out-of-sample return distribution, it can be written as

$$\mathrm{Omega}(\tau) = \frac{\displaystyle\sum_{k=1}^{T_{\mathrm{out}}} \max(r_{p,k} - \tau, 0)}{\displaystyle\sum_{k=1}^{T_{\mathrm{out}}} \max(\tau - r_{p,k}, 0)}. \tag{187}$$

The threshold convention used in the numerical implementation should be reported together with the corresponding results because the source manuscript defines Omega relative to a specified threshold but does not provide its numerical value in the evaluation-metrics section.

9.2.4 Mean Return

The mean out-of-sample portfolio return is

$$\overline{r}_p = \widehat{\mu}_p = \frac{1}{T_{\text{out}}} \sum_{k=1}^{T_{\text{out}}} r_{p,k}. \tag{188}$$

The numerical tables in the source study report this quantity under the label "Return mean."

9.2.5 Alpha and Beta

Alpha and beta are evaluated relative to the selected market benchmark. Let $r_{m,k}$ denote the benchmark return. The portfolio beta is

$$\beta_p = \frac{\mathrm{Cov}(r_p, r_m)}{\mathrm{Var}(r_m)}. \tag{189}$$

With a zero risk-free rate, the corresponding regression/intercept formulation for alpha is

$$r_{p,k} = \alpha_p + \beta_p r_{m,k} + \varepsilon_k, \tag{190}$$

where α_p is the intercept of the portfolio-return regression on the benchmark return.

Equivalently,

$$\alpha_p = \widehat{\mu}_p - \beta_p \widehat{\mu}_m, \tag{191}$$

under the zero risk-free-rate convention.

The source study uses the S&P 500 as the external market benchmark and reports Alpha and Beta for each portfolio strategy.

9.3 Portfolio Composition Metrics

In addition to return and risk characteristics, the analysis evaluates the concentration and trading intensity of the resulting portfolios.

9.3.1 Herfindahl–Hirschman Index

Portfolio concentration is measured using the Herfindahl–Hirschman Index (HHI):

$$\mathrm{HHI}_k = \sum_{i=1}^{n} \left(w_{i,k}^{\star}\right)^2. \tag{192}$$

For a fully invested long-only portfolio,

$$\frac{1}{n} \leq \mathrm{HHI}_k \leq 1. \tag{193}$$

A larger HHI corresponds to a more concentrated portfolio, whereas a value closer to $1/n$ corresponds to a more evenly distributed allocation. The source manuscript uses HHI specifically as its concentration measure.

9.3.2 Portfolio Turnover

For consecutive rebalancing dates $k-1$ and k, portfolio turnover is defined as

$$\mathrm{Turnover}_k = \sum_{i=1}^{n} \left| w_{i,k}^{\star} - w_{i,k-1}^{\star} \right|. \tag{194}$$

This measure captures the total absolute change in portfolio weights between two consecutive portfolio decisions.

For a buy-and-hold strategy, there is no periodic rebalancing after the initial allocation; consequently, turnover is not interpreted in the same way as for the periodic rebalancing experiments.

9.3.3 Number of Active Assets

The number of active assets is

$$N_{\text{assets},k} = \sum_{i=1}^{n} \mathbb{I}\left(w_{i,k}^{\star} \neq 0\right), \tag{195}$$

where $\mathbb{I}(\cdot)$ is the indicator function.

This metric measures the breadth of the portfolio at each rebalancing date.

9.4 Evaluation Across Rebalancing Frequencies

The evaluation is conducted under four periodic rebalancing frequencies:

$$R \in \{21, 63, 126, 252\}, \tag{196}$$

corresponding approximately to monthly, quarterly, semi-annual, and annual rebalancing, respectively. The source study also reports a buy-and-hold configuration.

For each rebalancing frequency, the same out-of-sample period is used for the competing strategies within a given experiment. This allows risk, return, and portfolio-composition measures to be compared under the same temporal evaluation period.

9.5 Evaluation Protocol

The evaluation procedure follows the temporal sequence

$$\boxed{\text{Historical data} \rightarrow \text{Model training} \rightarrow \text{Portfolio construction} \rightarrow \text{Future realized returns} \rightarrow \text{Performance metrics}} \tag{197}$$

In particular, none of the evaluation metrics is calculated from the latent representation itself. The Graph Transformer representation is used to estimate the expected-return vector through the return decoder, while all realized portfolio-performance metrics are calculated from the subsequent out-of-sample returns.

The distinction between the model quantities and the evaluation quantities is therefore

$$\begin{aligned} H_t &= \text{latent asset representation}, \\ \widehat{\boldsymbol{\mu}}_t &= \text{predicted expected-return vector}, \\ \widehat{\boldsymbol{\Sigma}}_t &= \text{historical return covariance matrix}, \\ \boldsymbol{w}_t^{\star} &= \text{Markowitz portfolio weights}, \\ r_{p,t+1} &= \text{realized out-of-sample portfolio return}. \end{aligned} \tag{198}$$

Consequently, the risk and performance metrics reported in the numerical analysis are calculated only after the corresponding portfolio returns have been realized. This preserves the chronological separation between model estimation, portfolio construction, and out-of-sample evaluation.

10 Numerical Results and Empirical Analysis

This section presents the empirical evaluation of the portfolio framework using the experimental configurations described in Section 8. The analysis considers different asset universes, historical training-window lengths, and portfolio rebalancing frequencies.

A methodological distinction is important for interpreting the numerical results. The tables inherited from the source implementation report the representation-based strategy under the label `MK(Embedding)`. In those experiments, the latent representation was directly incorporated

into the portfolio construction procedure. The revised formulation proposed in this manuscript replaces that step with the learned return decoder described in Section 5. Therefore, the numerical values reported below should be regarded as the results of the original implementation and not as numerical results of the revised decoder-based model.

Consequently, these results are retained as a reference for the empirical structure of the original study, whereas the final numerical tables of the revised manuscript must be regenerated after retraining the proposed model.

10.1 Experimental Configurations

The empirical analysis in the source study considers four principal configurations.

The first configuration uses a one-year training window of

$$T_{\text{outer}} = 252 \tag{199}$$

trading days and evaluates the model on the S&P 100 universe.

The remaining three configurations use the DJIA universe and consider training windows of

$$T_{\text{outer}} \in \{252, 504, 756\}, \tag{200}$$

corresponding approximately to one-, two-, and three-year historical windows.

The out-of-sample evaluation period is based on observations from 2021 to 2026. The one-year S&P 100 configuration contains 98 available assets, whereas the one-year DJIA configuration contains 30 assets.

For each configuration, five portfolio implementation schemes are considered:

1. buy and hold;
2. monthly rebalancing;
3. quarterly rebalancing;
4. semi-annual rebalancing;
5. annual rebalancing.

The periodic rebalancing intervals correspond to 21, 63, 126, and 252 trading days, respectively.

10.2 S&P 100: One-Year Training Window

For the one-year S&P 100 configuration, the source study uses 252 trading days for the training window and 98 available constituent assets. The reported comparison includes `MK(Embedding)`, conventional `MK`, equal weighting (`EW`), and the S&P 500 benchmark.

Under buy and hold, the reported values are shown in Table 2.

The source results show that the representation-based formulation and conventional Markowitz strategy exhibit substantially different realized risk characteristics. In particular, the reported volatility and maximum drawdown of `MK(Embedding)` are lower than those of conventional `MK`, while the passive strategies exhibit still lower volatility in this configuration.

Under monthly rebalancing, the source reports a volatility of 0.015960 for `MK(Embedding)` compared with 0.028194 for `MK`. The corresponding Sharpe ratios are 0.098597 and 0.046859, respectively.

Under quarterly rebalancing, the reported mean returns are 0.001449 for `MK(Embedding)` and 0.001698 for `MK`, while the corresponding volatilities are 0.015115 and 0.028204.

For semi-annual and annual rebalancing, the reported `MK(Embedding)` volatilities are 0.015311 and 0.014699, respectively, compared with 0.025557 and 0.024716 for conventional `MK`.

Table 2: Original reported results for the 252-day S&P 100 configuration under buy and hold.

Metric	MK(Embedding)	MK	EW	S&P500
Volatility	0.012150	0.022411	0.009367	0.010716
Max Drawdown	-0.229593	-0.514608	-0.203667	-0.254251
$\text{VaR}_{0.95}$	-0.017282	-0.035130	-0.014252	-0.016463
$\text{CVaR}_{0.95}$	-0.027368	-0.050702	-0.021321	-0.024565
Sharpe	0.051713	0.024200	0.051104	0.044585
Sortino	0.074235	0.035588	0.071647	0.061606
Omega	1.139717	1.060650	1.131702	1.114823
Alpha	3.015969×10^{-4}	-1.111141×10^{-4}	1.019438×10^{-4}	≈ 0
Beta	0.683826	1.367709	0.788524	1.000000
Return mean	0.000628	0.000542	0.000479	0.000478

10.3 Portfolio Concentration and Turnover

The source study also reports HHI and turnover for the 252-day S&P 100 configuration. The results are reproduced in Table 3.

Table 3: Original reported HHI and turnover for the 252-day S&P 100 configuration.

Rebalancing	MK HHI	MK(Embedding) HHI	MK Turnover	MK(Embedding) Turnover
Monthly	0.717191	0.355576	0.829382	0.888887
Quarterly	0.701123	0.357691	1.310127	1.074876
Semi-Annual	0.693727	0.361132	1.722535	1.267030
Annual	0.767362	0.325870	1.811572	1.003029

These values indicate that the source representation-based strategy generated lower HHI values than conventional Markowitz across all reported rebalancing frequencies, while the turnover relationship depended on the rebalancing frequency.

10.4 DJIA: One-Year Training Window

The one-year DJIA configuration uses 252 trading days for training, a 2021–2026 out-of-sample period, and 30 constituent assets. Tables 7–11 of the source study report the corresponding results for buy and hold and the four periodic rebalancing frequencies.

For example, under annual rebalancing, the reported results are

Table 4: Original reported results for the 252-day DJIA configuration under annual rebalancing.

Metric	MK(Embedding)	MK	EW	S&P500
Volatility	0.015747	0.024568	0.009566	0.010714
Max Drawdown	-0.134249	-0.286647	-0.110098	-0.118487
$\text{VaR}_{0.95}$	-0.022554	-0.034364	-0.013933	-0.016463
$\text{CVaR}_{0.95}$	-0.031500	-0.053261	-0.021593	-0.024565
Sharpe	0.090309	0.064271	0.058527	0.045168
Sortino	0.147672	0.094311	0.082184	0.062401
Omega	1.262665	1.189956	1.157158	1.116635
Alpha	8.733195×10^{-4}	8.604640×10^{-4}	1.582531×10^{-4}	≈ 0
Beta	1.133976	1.484776	0.829933	1.000000
Return mean	0.001422	0.001579	0.000560	0.000484

The associated HHI and turnover values are

Table 5: Original reported HHI and turnover for the 252-day DJIA configuration.

Rebalancing	MK HHI	MK(Embedding) HHI	MK Turnover	MK(Embedding) Turnover
Monthly	0.805086	0.469475	0.451776	0.920522
Quarterly	0.759489	0.393553	0.845491	0.782771
Semi-Annual	0.702299	0.401500	1.198948	0.751075
Annual	0.793066	0.367591	1.261091	0.784518

The source results therefore show a systematic difference in portfolio concentration between the two Markowitz formulations in this particular experiment. The turnover comparison, however, changes with the rebalancing interval: the reported representation-based strategy has higher turnover under monthly rebalancing but lower turnover under quarterly, semi-annual, and annual rebalancing.

10.5 DJIA: Two-Year Training Window

The second DJIA configuration uses

$$T_{\text{outer}} = 504 \tag{201}$$

trading days, corresponding approximately to a two-year historical training window. The training period covers 2019–2021 and the out-of-sample period extends from 2021 to 2026. The universe contains 30 assets.

Under buy and hold, the source reports

Table 6: Original reported results for the 504-day DJIA configuration under buy and hold.

Metric	MK(Embedding)	MK	EW	S&P500
Volatility	0.019360	0.032877	0.009583	0.010740
Max Drawdown	-0.507029	-0.663636	-0.226788	-0.254251
$\text{VaR}_{0.95}$	-0.028775	-0.048963	-0.013951	-0.016463
$\text{CVaR}_{0.95}$	-0.039460	-0.067164	-0.021593	-0.024565
Sharpe	0.075996	0.073156	0.057328	0.044374
Sortino	0.123720	0.118635	0.080534	0.061393
Omega	1.219375	1.214595	1.156484	1.116745
Alpha	8.327409×10^{-4}	1.362775×10^{-3}	1.540404×10^{-4}	≈ 0
Beta	1.339909	2.187312	0.829513	1.000000
Return mean	0.001471	0.002405	0.000549	0.000477

The results show that extending the training window changes both the realized return and risk characteristics of the portfolio strategies. In particular, under the buy-and-hold configuration, the reported maximum drawdowns are substantially larger than those observed in the corresponding one-year DJIA configuration.

For the periodic-rebalancing experiments, the source reports the corresponding risk, performance, HHI, and turnover measures in Tables 14–18. The reported HHI and turnover values for the 504-day configuration are

Table 7: Original reported HHI and turnover for the 504-day DJIA configuration.

Rebalancing	MK HHI	MK(Embedding) HHI	MK Turnover	MK(Embedding) Turnover
Monthly	0.704511	0.423638	0.396339	0.440852
Quarterly	0.740876	0.442102	0.613165	0.660296
Semi-Annual	0.765862	0.448773	0.649967	0.404959
Annual	0.768240	0.461928	0.917294	0.455669

These values illustrate that the concentration and turnover characteristics are sensitive to the historical training-window length and the rebalancing frequency.

10.6 DJIA: Three-Year Training Window

The third DJIA configuration uses a 756-day training window, corresponding approximately to three years. The training period covers 2018–2021, followed by out-of-sample evaluation from 2021 to 2026. The source reports 29 assets for this configuration.

For example, under semi-annual rebalancing, the reported results are

Table 8: Original reported results for the 756-day DJIA configuration under semi-annual rebalancing.

Metric	MK(Embedding)	MK	EW	S&P500
Volatility	0.016772	0.026927	0.009520	0.010716
Max Drawdown	-0.158448	-0.264115	-0.106424	-0.118487
$\mathrm{VaR}_{0.95}$	-0.023733	-0.041929	-0.014017	-0.016463
$\mathrm{CVaR}_{0.95}$	-0.032999	-0.057238	-0.021546	-0.024565
Sharpe	0.085208	0.064080	0.060620	0.044585
Sortino	0.142328	0.098856	0.085057	0.061606
Omega	1.257211	1.185904	1.169147	1.119470
Alpha	8.925353×10^{-4}	8.207097×10^{-4}	1.808808×10^{-4}	≈ 0
Beta	1.123131	1.893752	0.829239	1.000000
Return mean	0.001429	0.001726	0.000577	0.000478

For the annual rebalancing configuration, the reported values are

$$\begin{aligned} \mathrm{Volatility}_{\mathrm{MK(Embedding)}} &= 0.016526, \\ \mathrm{Volatility}_{\mathrm{MK}} &= 0.024878, \\ \mathrm{Sharpe}_{\mathrm{MK(Embedding)}} &= 0.075295, \\ \mathrm{Sharpe}_{\mathrm{MK}} &= 0.071013. \end{aligned} \tag{202}$$

The corresponding mean returns are 0.001244 and 0.001767, respectively.

The HHI and turnover results for the three-year configuration are

Table 9: Original reported HHI and turnover for the 756-day DJIA configuration.

Rebalancing	MK HHI	MK(Embedding) HHI	MK Turnover	MK(Embedding) Turnover
Monthly	0.670112	0.472085	0.296772	0.268032
Quarterly	0.665518	0.464294	0.507914	0.282118
Semi-Annual	0.685930	0.497247	0.533753	0.291262
Annual	0.639779	0.480955	0.592361	0.345292

The source study reports that, in this three-year configuration, the representation-based strategy has lower HHI and lower turnover than conventional Markowitz across all four periodic rebalancing frequencies.

10.7 Cross-Configuration Observations

Across the reported experiments, several empirical patterns are visible in the original implementation.

First, the reported volatility and downside-risk measures of `MK(Embedding)` are generally lower than those of conventional `MK` under periodic rebalancing. This pattern appears in the one-year DJIA configuration and remains visible in the two- and three-year configurations.

Second, the relationship with the passive benchmarks is different. The reported EW and S&P 500 portfolios frequently exhibit lower absolute volatility and tail-risk measures than both Markowitz-based strategies. Thus, lower risk relative to conventional Markowitz should not be interpreted as minimum absolute risk across all strategies.

Third, the reported risk-adjusted metrics, including Sharpe, Sortino, and Omega, vary with both the training-window length and the rebalancing frequency. For example, in the three-year DJIA experiment, the reported Sharpe ratios of `MK(Embedding)` remain above those of conventional `MK` for the four periodic rebalancing configurations.

Fourth, portfolio concentration differs substantially between the two Markowitz formulations in the reported experiments. The HHI values of `MK(Embedding)` are lower than those of conventional `MK` across all four rebalancing frequencies in the three-year DJIA configuration.

Finally, turnover is not uniformly lower for the representation-based strategy. In the one-year DJIA experiment, for example, `MK(Embedding)` has higher turnover under monthly rebalancing but lower turnover under the longer rebalancing intervals.

10.8 Interpretation in the Revised Framework

The empirical results above describe the original implementation and therefore cannot by themselves establish the performance of the revised decoder-based formulation.

In the revised framework, the information flow is

$$G_t \to H_t \to \widehat{\boldsymbol{\mu}}_t \to \boldsymbol{w}_t^\star. \tag{203}$$

Thus, the quantity entering the Markowitz objective is the predicted return vector

$$\widehat{\boldsymbol{\mu}}_t = \left(\widehat{r}_{1,t+1}, \ldots, \widehat{r}_{n,t+1}\right)^\top, \tag{204}$$

rather than an aggregation of latent embedding coordinates.

Accordingly, the final numerical experiments of the revised manuscript should retrain the model under the new joint objective

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}} \tag{205}$$

and regenerate the portfolio weights and all out-of-sample performance measures.

The existing tables should therefore be treated as a baseline reference rather than as evidence for the revised model.

10.9 Required Numerical Re-Evaluation

For the revised model, the following quantities must be regenerated:

1. predicted expected returns $\widehat{\boldsymbol{\mu}}_t$ from the return decoder;
2. covariance matrices $\widehat{\boldsymbol{\Sigma}}_t$ from realized historical returns;
3. Markowitz weights $\boldsymbol{w}_t^\star$;
4. realized out-of-sample portfolio returns;
5. volatility and maximum drawdown;
6. VaR and CVaR;
7. Sharpe, Sortino, and Omega ratios;
8. Alpha and Beta;

9. mean portfolio return;
10. HHI;
11. turnover; and
12. number of active assets.

The same evaluation horizons and rebalancing frequencies should be retained so that the revised results remain directly comparable with the original experimental design.

In particular, no table should label results obtained from the revised decoder-based model as "MK(Embedding)" if the portfolio expected returns are generated by the learned return decoder. A distinct label such as

$$\text{MK(GT-Decoder)} \tag{206}$$

may be used to distinguish the revised model from the original implementation.

This distinction is necessary because the central methodological change of this manuscript is not merely an architectural modification of the Graph Transformer. It changes the statistical interpretation of the quantity supplied to the Markowitz optimization problem.

11 Discussion

The proposed framework combines financial graph construction, an edge-aware Graph Transformer, contrastive representation learning, supervised return prediction, and Markowitz portfolio optimization. The main methodological contribution of the revised framework is the explicit separation between the latent representation learned by the Graph Transformer and the financial quantities required by portfolio optimization.

11.1 Role of the Graph Transformer Representation

The Graph Transformer produces an asset-level representation

$$H_t = \begin{bmatrix} h_{1,t}^\top \\ \vdots \\ h_{n,t}^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \tag{207}$$

where $h_{i,t} \in \mathbb{R}^d$ summarizes information extracted from the financial graph.

The representation incorporates both node-level return information and pairwise relationships between assets. In particular, the edge attributes allow the attention mechanism to condition message passing on several dependence measures. Consequently, the representation is not restricted to the historical return sequence of an individual asset but is influenced by its position and relationships within the financial network.

This property is particularly relevant for portfolio construction because the investment opportunity of an asset cannot generally be considered independently of the other assets in the investment universe. Cross-asset dependence can affect both the expected behavior of an asset and the diversification properties of the resulting portfolio.

Nevertheless, the latent representation itself does not have the statistical interpretation of an expected return. The components of $h_{i,t}$ are learned latent variables, and their numerical scale and orientation are determined by the optimization problem rather than by the units of financial return.

11.2 Why the Return Decoder Is Necessary

A central modification of the revised framework is the introduction of an explicit return-prediction mapping

$$\widehat{r}_{i,t+1} = g_\phi(h_{i,t}). \tag{208}$$

The resulting vector

$$\widehat{\boldsymbol{\mu}}_t = \begin{bmatrix} \widehat{r}_{1,t+1} \\ \vdots \\ \widehat{r}_{n,t+1} \end{bmatrix} \tag{209}$$

has a direct financial interpretation and can therefore be supplied to the Markowitz optimization problem.

This distinction eliminates a conceptual inconsistency that arises when an arbitrary scalar function of the latent coordinates is interpreted directly as an expected return. For example, the quantity

$$z_{i,t} = \frac{1}{d} \sum_{k=1}^{d} H_{t,i,k} \tag{210}$$

is a summary of the latent representation, but there is no general theoretical reason for it to be measured in return units or to estimate

$$\mathbb{E}[r_{i,t+1} \mid G_t]. \tag{211}$$

The decoder instead learns this relationship through the supervised prediction objective. Thus,

$$H_t \quad \longrightarrow \quad \widehat{\boldsymbol{\mu}}_t \tag{212}$$

has an explicit statistical meaning.

11.3 Separation of Expected Return and Covariance Estimation

The revised framework also maintains a clear distinction between expected-return estimation and covariance estimation.

The expected-return vector is obtained from the learned prediction model:

$$\widehat{\boldsymbol{\mu}}_t = g_\phi(H_t), \tag{213}$$

whereas the covariance matrix is estimated from historical realized returns:

$$\widehat{\boldsymbol{\Sigma}}_t = \frac{1}{T_w - 1} \sum_{s=t-T_w+1}^{t} (r_s - \bar{r}_t)(r_s - \bar{r}_t)^\top. \tag{214}$$

Therefore, the two inputs to the portfolio optimization problem have different statistical origins:

$$\boxed{\widehat{\boldsymbol{\mu}}_t = \text{learned expected-return estimate}} \tag{215}$$

and

$$\boxed{\widehat{\boldsymbol{\Sigma}}_t = \text{historical return covariance estimate.}} \tag{216}$$

This separation also makes the interpretation of the resulting portfolio more transparent. The Graph Transformer is responsible for learning a representation useful for return prediction, while the covariance matrix continues to describe the observed co-movement of asset returns.

11.4 Interaction Between Contrastive Learning and Return Prediction

The revised training procedure combines two learning objectives:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}}. \tag{217}$$

The predictive loss encourages the representation to retain information that is useful for forecasting the next-period return. The contrastive loss imposes an additional structural constraint by encouraging representations of the same asset under different graph perturbations to remain similar while separating representations associated with different assets.

These two objectives serve different purposes.

The predictive objective is directly related to the downstream financial quantity:

$$r_{i,t+1}. \tag{218}$$

The contrastive objective instead acts as a representation-learning regularizer. It does not define an expected return and does not enter the Markowitz objective directly.

Consequently, the role of contrastive learning should be interpreted as improving the structure and robustness of the latent representation rather than replacing the supervised return-estimation task.

11.5 Importance of Graph Augmentation

The revised contrastive-learning formulation uses two perturbed views of the financial graph:

$$G_t^{(1)} = \mathcal{T}_1(G_t), \qquad G_t^{(2)} = \mathcal{T}_2(G_t). \tag{219}$$

The two transformations may involve edge dropout and feature masking. The shared Graph Transformer then produces

$$H_t^{(1)} = f_\theta(G_t^{(1)}), \qquad H_t^{(2)} = f_\theta(G_t^{(2)}). \tag{220}$$

The contrastive objective encourages the representation of the same asset to remain stable under these perturbations.

This is particularly relevant in a financial graph because pairwise dependence measures can vary over time. A representation that is excessively sensitive to small changes in individual edges may lead to unstable downstream return predictions and portfolio weights. The contrastive objective provides a mechanism for encouraging local representation stability.

At the same time, the augmentation strength must be chosen carefully. Excessive edge removal or feature masking can eliminate economically relevant information from the graph. Therefore, augmentation parameters should be regarded as model hyperparameters rather than as universally optimal constants.

11.6 Implications for Portfolio Optimization

After return prediction, the portfolio optimization problem is formulated as

$$\begin{aligned} \max_{\boldsymbol{w}_t} \quad & \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w}_t - \frac{\gamma}{2} \boldsymbol{w}_t^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w}_t \\ \text{s.t.} \quad & \mathbf{1}^\top \boldsymbol{w}_t = 1, \\ & \boldsymbol{w}_t \geq \mathbf{0}. \end{aligned} \tag{221}$$

The Graph Transformer therefore does not replace the Markowitz model. Instead, it provides a learned estimate of one of the two principal inputs to the mean-variance optimization problem.

The resulting architecture can be summarized as

$$\boxed{\text{Financial Returns} \rightarrow \text{Financial Graph} \rightarrow \text{Graph Transformer} \rightarrow \text{Asset Representation} \rightarrow \text{Return Decoder} \rightarrow \hat{}} \tag{222}$$

This formulation preserves the interpretable structure of the Markowitz framework while allowing the expected-return component to incorporate nonlinear and relational information extracted from the financial graph.

11.7 Effect of the Training-Window Length

The framework allows the outer training horizon to vary while maintaining a shorter rolling graph-construction window. Let

$$T_{\text{outer}} \in \{252, 504, 756\}. \tag{223}$$

Increasing T_{outer} provides the learning model with more historical observations, but it may also introduce observations generated under different market regimes. Therefore, a longer training window does not necessarily imply a uniformly better representation or more accurate return predictions.

The inner graph window T_w determines the observations used to construct the current financial graph, whereas the outer training window determines the historical information available for estimating the model parameters. These two time scales should therefore not be conflated.

The empirical analysis should consequently examine the interaction between

$$T_{\text{outer}}, \qquad T_w, \qquad \text{and the rebalancing frequency.} \tag{224}$$

Such an analysis can help determine whether additional historical information improves the stability of the learned mapping

$$G_t \rightarrow \widehat{\boldsymbol{\mu}}_t \tag{225}$$

or instead introduces sensitivity to older market conditions.

11.8 Portfolio Concentration and Turnover

The portfolio-level consequences of the learned expected returns cannot be evaluated solely through mean return or volatility. The Markowitz objective may generate highly concentrated allocations when estimated expected returns differ substantially across assets.

For this reason, the revised evaluation includes both the Herfindahl–Hirschman Index

$$\text{HHI}_t = \sum_{i=1}^{n} w_{i,t}^2 \tag{226}$$

and portfolio turnover

$$\text{Turnover}_t = \sum_{i=1}^{n} \left| w_{i,t} - w_{i,t-1} \right|. \tag{227}$$

The HHI measures the concentration of the portfolio, whereas turnover measures the magnitude of changes in portfolio positions between consecutive rebalancing dates.

These two quantities capture different aspects of portfolio behavior. A portfolio can be relatively diversified while still undergoing frequent changes, or it can remain concentrated while exhibiting relatively low turnover. Consequently, both measures should be reported separately.

11.9 Interpretation of the Empirical Comparison

The comparison between the revised Graph Transformer–Markowitz framework and conventional Markowitz optimization should focus on the source of the difference between the two approaches.

In conventional Markowitz optimization, the expected-return vector is specified according to the baseline estimation procedure. In the proposed framework, the corresponding vector is generated by

$$\widehat{\boldsymbol{\mu}}_t = g_\phi!\left(f_\theta(G_t)\right). \tag{228}$$

Thus, any difference in portfolio performance should be interpreted through the effect of the learned expected-return estimates on the optimization problem.

The appropriate empirical questions are therefore:

1. Does the Graph Transformer improve out-of-sample return prediction?
2. Does the learned return vector produce different portfolio allocations from the conventional Markowitz specification?
3. How does the revised model affect portfolio volatility and downside risk?
4. How does the model affect concentration and turnover?
5. Are the observed effects stable across asset universes and training-window lengths?
6. How sensitive are the results to the rebalancing frequency?

These questions provide a more direct evaluation of the proposed methodology than interpreting latent embedding coordinates themselves as financial returns.

11.10 Limitations

Several limitations should be considered when interpreting the framework.

First, the quality of the portfolio allocation depends on the quality of the estimated expected-return vector. Expected returns are difficult to estimate accurately, and prediction errors can have a substantial effect on mean-variance optimization.

Second, the Markowitz objective is sensitive to estimation error in both expected returns and covariance matrices. Although the revised framework improves the interpretation of the expected-return component, it does not eliminate estimation uncertainty.

Third, the financial graph is constructed from historical dependence measures. These relationships are time-varying, and a graph estimated over a finite rolling window may not fully represent future dependence structures.

Fourth, the contrastive-learning objective introduces additional hyperparameters, including the augmentation probabilities, temperature, and loss weight. Their effect should be investigated through sensitivity analysis.

Finally, the revised methodology requires a complete re-estimation of the empirical results. The numerical results inherited from the original implementation should not be interpreted as validation of the decoder-based formulation until the revised model has been trained and evaluated under the same out-of-sample protocol.

11.11 Summary of the Revised Methodological Interpretation

The principal conceptual distinction of the revised framework can be summarized as

$$\boxed{\begin{aligned} H_t &= \text{learned latent asset representation},\\ \widehat{\boldsymbol{\mu}}_t &= \text{decoder-based expected-return estimate},\\ \widehat{\boldsymbol{\Sigma}}_t &= \text{historical return covariance},\\ \boldsymbol{w}_t^\star &= \text{Markowitz portfolio weights},\\ r_{p,t+1} &= \text{realized portfolio return}. \end{aligned}} \tag{229}$$

This separation gives each quantity a well-defined role in the proposed architecture. The Graph Transformer learns relational representations from the financial graph, the return decoder maps those representations into the return space, and the Markowitz optimizer converts the resulting expected-return estimates and historical covariance information into portfolio decisions.

12 Conclusion

This study developed a graph-based portfolio optimization framework that integrates financial graph construction, an edge-aware Graph Transformer, contrastive representation learning, supervised expected-return estimation, and Markowitz portfolio optimization.

The financial graph represents assets as nodes and incorporates multiple pairwise dependence measures as edge attributes. The Graph Transformer then learns asset-level representations by combining node information with the relational structure of the financial network. This allows the representation of each asset to depend not only on its own historical return information but also on its relationships with other assets in the investment universe.

A central methodological point of the revised framework is the distinction between a learned latent representation and an expected return. The latent vector

$$h_{i,t} \in \mathbb{R}^d \tag{230}$$

is an internal representation learned by the Graph Transformer and does not have an intrinsic interpretation as a financial return. Therefore, rather than converting the latent vector into a scalar by an arbitrary aggregation, the proposed framework introduces an explicit return decoder:

$$\widehat{r}_{i,t+1} = g_\phi(h_{i,t}). \tag{231}$$

The predicted expected-return vector is consequently defined as

$$\widehat{\boldsymbol{\mu}}_t = (\widehat{r}_{1,t+1}, \ldots, \widehat{r}_{n,t+1})^\top. \tag{232}$$

This formulation gives the quantity entering the Markowitz optimization problem a direct statistical interpretation. At the same time, the covariance matrix remains estimated from realized historical returns:

$$\widehat{\boldsymbol{\Sigma}}_t = \frac{1}{T_w - 1} \sum_{s=t-T_w+1}^{t} (r_s - \bar{r}_t)(r_s - \bar{r}_t)^\top. \tag{233}$$

The resulting portfolio optimization problem therefore combines a learned expected-return estimate with a covariance estimate defined in the original return space:

$$\max_{\boldsymbol{w}_t} \quad \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w}_t - \frac{\gamma}{2} \boldsymbol{w}_t^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w}_t, \tag{234}$$

subject to the portfolio constraints.

The contrastive-learning component provides an additional representation-learning objective. By constructing two perturbed views of the same financial graph and encouraging consistent representations for corresponding assets, the model is designed to learn representations that are less sensitive to individual graph perturbations. Importantly, this objective is auxiliary: it supports representation learning but does not replace the supervised return-prediction objective.

The complete information flow of the revised framework can therefore be summarized as

$$\boxed{\text{Historical Returns} \rightarrow \text{Financial Graph} \rightarrow \text{Graph Transformer} \rightarrow \text{Latent Representation} \rightarrow \text{Return Decoder} \rightarrow} \tag{235}$$

The empirical design considers different asset universes, historical training-window lengths, and portfolio rebalancing frequencies. These dimensions allow the behavior of the proposed framework to be evaluated under different temporal and portfolio-management settings. In addition to conventional return and risk measures, portfolio concentration and turnover provide information about the practical characteristics of the resulting allocations.

The numerical results currently available from the source implementation were generated using the original representation-based formulation. They therefore provide a reference for the empirical design but should not be interpreted as numerical validation of the revised return-decoder formulation. A definitive empirical assessment of the revised framework requires retraining the encoder, contrastive-learning module, and return decoder and then regenerating the out-of-sample portfolio results under the same evaluation protocol.

The revised formulation also provides a clearer basis for future extensions. In particular, future work can investigate alternative return-decoder architectures, more robust covariance estimators, transaction costs, portfolio constraints such as position limits, short-selling restrictions, and dynamic risk-aversion parameters. Sensitivity analysis with respect to graph-construction windows, contrastive-learning hyperparameters, and training horizons can also be used to examine the stability of the learned return estimates.

Overall, the methodological framework establishes a clear division of roles between representation learning and portfolio optimization:

$$\boxed{\begin{aligned} &H_t = \text{relational latent representation},\\ &\widehat{\boldsymbol{\mu}}_t = \text{predicted expected returns},\\ &\widehat{\boldsymbol{\Sigma}}_t = \text{historical return covariance},\\ &\boldsymbol{w}_t^{\star} = \text{optimal portfolio weights}. \end{aligned}} \tag{236}$$

This separation is essential for maintaining the financial interpretation of the optimization problem while exploiting the nonlinear and relational information captured by Graph Transformer representations.

A Mathematical Details and Training Algorithm

This appendix provides the mathematical details of the Graph Transformer, contrastive-learning objective, return decoder, and rolling-window training procedure used in the proposed framework.

A.1 Edge-Aware Graph Transformer

Let

$$G_t = (V, E_t, X_t, A_t) \tag{237}$$

denote the financial graph at time t, where X_t contains node features and A_t contains edge attributes. For an asset i, let $x_{i,t}$ denote its node feature vector. For an edge $(i, j) \in E_t$, let $a_{ij,t}$ denote its edge-attribute vector.

For a single attention head, the edge-aware attention mechanism can be written as

$$q_{i,t} = W_Q x_{i,t}, \tag{238}$$

$$k_{ij,t} = W_K x_{j,t} + W_E a_{ij,t}, \tag{239}$$

and

$$v_{ij,t} = W_V x_{j,t} + W_{EV} a_{ij,t}. \tag{240}$$

The attention coefficient associated with edge (i, j) is

$$\alpha_{ij,t} = \frac{\exp\left(q_{i,t}^\top k_{ij,t}/\sqrt{d_h}\right)}{\sum_{\ell \in \mathcal{N}_t(i)} \exp\left(q_{i,t}^\top k_{i\ell,t}/\sqrt{d_h}\right)}, \tag{241}$$

where d_h denotes the dimension of the attention head and $\mathcal{N}_t(i)$ denotes the neighborhood of node i.

The output of the attention head is

$$u_{i,t} = W_O \left(\sum_{j \in \mathcal{N}_t(i)} \alpha_{ij,t} v_{ij,t} \right). \tag{242}$$

For K attention heads, the outputs can be concatenated or averaged according to the selected Graph Transformer architecture. The resulting layer output is denoted by

$$H_t^{(\ell)} = \mathrm{GT}^{(\ell)}\left(H_t^{(\ell-1)}, A_t\right), \tag{243}$$

with

$$H_t^{(0)} = X_t. \tag{244}$$

Residual connections, layer normalization, nonlinear activation, and dropout are applied according to the implemented architecture. After the final Graph Transformer layer,

$$H_t = H_t^{(L)} = \begin{bmatrix} h_{1,t}^\top \\ \vdots \\ h_{n,t}^\top \end{bmatrix} \in \mathbb{R}^{n \times d}. \tag{245}$$

Each vector $h_{i,t}$ represents the learned relational representation of asset i at time t.

A.2 Contrastive Learning Objective

For each graph G_t, two stochastic graph views are generated:

$$G_t^{(1)} = \mathcal{T}_1(G_t), \qquad G_t^{(2)} = \mathcal{T}_2(G_t), \tag{246}$$

where the transformations may include edge dropout and node-feature masking.

The same Graph Transformer encoder is applied to both views:

$$H_t^{(1)} = f_\theta(G_t^{(1)}), \qquad H_t^{(2)} = f_\theta(G_t^{(2)}). \tag{247}$$

A projection head $g_\psi(\cdot)$ maps the representations into the contrastive space:

$$z_{i,t}^{(v)} = g_\psi\left(h_{i,t}^{(v)}\right), \qquad v \in \{1, 2\}. \tag{248}$$

The projected vectors are normalized as

$$\widetilde{z}_{i,t}^{(v)} = \frac{z_{i,t}^{(v)}}{\left\| z_{i,t}^{(v)} \right\|_2}. \tag{249}$$

For a positive pair consisting of the two views of the same asset, the similarity is measured by

$$s_{ij,t}^{(u,v)} = \frac{\left(\widetilde{z}_{i,t}^{(u)}\right)^{\top} \widetilde{z}_{j,t}^{(v)}}{\tau}, \tag{250}$$

where $\tau > 0$ is the temperature parameter.

The NT-Xent loss for asset i in view u with the corresponding asset in the other view as the positive example is

$$\ell_{i,t}^{(u,v)} = -\log \frac{\exp\left(s_{ii,t}^{(u,v)}\right)}{\sum_{j=1}^{n} \exp\left(s_{ij,t}^{(u,v)}\right)}. \tag{251}$$

The symmetric contrastive loss is

$$\mathcal{L}_{\mathrm{CL}} = \frac{1}{2n} \sum_{i=1}^{n} \left[\ell_{i,t}^{(1,2)} + \ell_{i,t}^{(2,1)}\right]. \tag{252}$$

Thus, the representation-learning objective encourages the two augmented views of the same asset to remain close while distinguishing them from representations of other assets.

A.3 Return Decoder

The latent representation generated by the Graph Transformer is transformed into a predicted next-period return through a learnable decoder:

$$\widehat{r}_{i,t+1} = g_{\phi}(h_{i,t}). \tag{253}$$

For a multilayer perceptron decoder, a general form is

$$g_{\phi}(h) = W_2 \sigma(W_1 h + b_1) + b_2, \tag{254}$$

where $\sigma(\cdot)$ denotes a nonlinear activation function.

The return-prediction loss is defined as

$$\mathcal{L}_{\mathrm{pred}} = \frac{1}{n} \sum_{i=1}^{n} \left(r_{i,t+1} - \widehat{r}_{i,t+1}\right)^2. \tag{255}$$

The complete training objective is

$$\mathcal{L} = \mathcal{L}_{\mathrm{pred}} + \lambda_{\mathrm{CL}} \mathcal{L}_{\mathrm{CL}}, \tag{256}$$

where $\lambda_{\mathrm{CL}} \geq 0$ controls the contribution of the contrastive-learning objective.

The two components have distinct roles:

$$\mathcal{L}_{\mathrm{pred}} \quad \text{provides financial prediction supervision,} \tag{257}$$

while

$$\mathcal{L}_{\mathrm{CL}} \quad \text{regularizes the learned relational representation.} \tag{258}$$

A.4 Expected Return and Covariance Estimation

After training on the information available at time t, the decoder produces the expected-return estimate

$$\widehat{\boldsymbol{\mu}}_t = \begin{bmatrix} \widehat{r}_{1,t+1} \\ \vdots \\ \widehat{r}_{n,t+1} \end{bmatrix}. \tag{259}$$

The covariance matrix is estimated directly from the historical return observations in the rolling window:

$$\bar{r}_t = \frac{1}{T_w} \sum_{s=t-T_w+1}^{t} r_s, \tag{260}$$

and

$$\widehat{\boldsymbol{\Sigma}}_t = \frac{1}{T_w - 1} \sum_{s=t-T_w+1}^{t} (r_s - \bar{r}_t)(r_s - \bar{r}_t)^\top. \tag{261}$$

Consequently, the expected-return vector and covariance matrix are estimated in two different but complementary spaces:

$$\widehat{\boldsymbol{\mu}}_t \in \mathbb{R}^n \tag{262}$$

is obtained from the learned representation through the return decoder, whereas

$$\widehat{\boldsymbol{\Sigma}}_t \in \mathbb{R}^{n \times n} \tag{263}$$

is computed from realized historical returns.

A.5 Markowitz Optimization

Given $\widehat{\boldsymbol{\mu}}_t$ and $\widehat{\boldsymbol{\Sigma}}_t$, portfolio weights are obtained from

$$\boldsymbol{w}_t^\star = \arg\max_{\boldsymbol{w}} \left\{ \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w} - \frac{\gamma}{2} \boldsymbol{w}^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w} \right\}, \tag{264}$$

subject to

$$\mathbf{1}^\top \boldsymbol{w} = 1 \tag{265}$$

and any additional portfolio constraints imposed in the experimental setting.

The realized return in the subsequent period is

$$r_{p,t+1} = \boldsymbol{w}_t^{\star\top} r_{t+1}. \tag{266}$$

Importantly, $r_{p,t+1}$ is not used in constructing the portfolio at time t. It is used only for subsequent out-of-sample evaluation.

A.6 Rolling-Window Training Procedure

Let $\mathcal{I}_t$ denote all information available at the portfolio formation time t. The complete procedure is as follows.

1. Construct the rolling financial graph G_t using only observations contained in the available historical window.
2. Calculate the node features and edge attributes from the historical returns.
3. Generate two augmented graph views,

$$G_t^{(1)} = \mathcal{T}_1(G_t), \qquad G_t^{(2)} = \mathcal{T}_2(G_t). \tag{267}$$

4. Pass the two graph views through the shared Graph Transformer encoder.

5. Compute the contrastive loss from the corresponding asset representations.

6. Use the original graph representation to generate predicted returns through the return decoder.

7. Calculate the prediction loss using the appropriate training targets available within the training sample.

8. Jointly optimize
$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}}\mathcal{L}_{\text{CL}}. \tag{268}$$

9. Generate
$$\widehat{\boldsymbol{\mu}}_t \tag{269}$$
using the trained decoder.

10. Estimate
$$\widehat{\boldsymbol{\Sigma}}_t \tag{270}$$
from historical returns.

11. Solve the Markowitz optimization problem and obtain
$$\boldsymbol{w}_t^{\star}. \tag{271}$$

12. Hold the resulting portfolio until the next scheduled rebalancing date.

13. Evaluate the realized portfolio return using observations that occur after portfolio formation.

A.7 Algorithmic Representation

The complete rolling procedure can be summarized as follows.

Algorithm 1 Graph Transformer–Decoder–Markowitz Portfolio Construction

Require: Historical asset returns, graph window T_w, training window T_{train}, risk-aversion parameter γ

Ensure: Portfolio weights $\boldsymbol{w}_t^{\star}$ for each rebalancing date

for each rebalancing date t **do**
  Construct G_t using information available up to t
  Generate two augmented views $G_t^{(1)}$ and $G_t^{(2)}$
  Encode both views using the shared Graph Transformer
  Compute $\mathcal{L}_{\text{CL}}$
  Generate predicted returns using the return decoder
  Compute $\mathcal{L}_{\text{pred}}$
  Minimize $\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}}\mathcal{L}_{\text{CL}}$
  Estimate $\widehat{\boldsymbol{\Sigma}}_t$ from historical returns
  Generate $\widehat{\boldsymbol{\mu}}_t$ using the trained decoder
  Solve the Markowitz optimization problem
  Obtain $\boldsymbol{w}_t^{\star}$
  Evaluate subsequent realized portfolio returns
end for

A.8 Information-Set Consistency

At every portfolio formation date, all quantities used to construct the portfolio must be measurable with respect to the information available at that date. In particular,

$$G_t, \quad H_t, \quad \widehat{\boldsymbol{\mu}}_t, \quad \widehat{\boldsymbol{\Sigma}}_t \tag{272}$$

must be constructed without using future observations.

The realized return

$$r_{t+1} \tag{273}$$

is observed only after the portfolio has been formed and is therefore reserved for out-of-sample evaluation.

This timing condition is essential because any use of future returns in graph construction, model training, expected-return estimation, covariance estimation, or portfolio optimization would introduce look-ahead bias and invalidate the out-of-sample evaluation.

B Hyperparameter Configuration and Reproducibility Details

This appendix summarizes the principal implementation and training parameters used in the experimental framework. The purpose is to make the computational procedure sufficiently explicit for reproducibility and to distinguish parameters associated with graph construction, representation learning, return prediction, and portfolio optimization.

B.1 Dataset and Return Construction

The empirical analysis considers the Dow Jones Industrial Average (DJIA) and the S&P 100 universes. Historical adjusted closing prices are used to calculate daily simple returns according to

$$r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}, \tag{274}$$

where $P_{i,t}$ denotes the adjusted closing price of asset i at trading day t.

For the one-year configuration, the DJIA universe contains 30 assets, while the S&P 100 universe contains 98 assets in the experimental sample. The main training and out-of-sample periods are defined according to the experimental design described in Section 8.

B.2 Rolling Windows

Three outer training-window lengths are considered:

$$T_{\text{train}} \in \{252, 504, 756\}. \tag{275}$$

These correspond approximately to one, two, and three years of trading observations.

Within each outer training period, the financial graph is constructed using an inner rolling window of

$$T_w = 20 \tag{276}$$

trading days.

The use of separate outer and inner windows distinguishes model-training history from the local historical information used to construct the financial graph and estimate the covariance matrix.

B.3 Financial Graph Features

Each graph contains node features derived from historical asset returns. Edge attributes are constructed from five pairwise measures:

1. Pearson correlation,
2. Spearman correlation,
3. normalized covariance,
4. Kendall correlation,
5. cosine similarity.

Thus, for each pair (i, j),

$$a_{ij,t} = \left[a_{ij,t}^{(1)}, a_{ij,t}^{(2)}, a_{ij,t}^{(3)}, a_{ij,t}^{(4)}, a_{ij,t}^{(5)}\right]^{\top}. \tag{277}$$

The graph is constructed as a complete directed graph, including self-pairs when they are retained by the implementation.

B.4 Graph Transformer Architecture

The Graph Transformer consists of three TransformerConv layers. The principal architectural configuration is

$$64 \to 64 \to 32, \tag{278}$$

where the first two layers use 64 hidden features and four attention heads, while the final layer produces a 32-dimensional asset representation.

Accordingly,

$$h_{i,t} \in \mathbb{R}^{32}. \tag{279}$$

Residual connections, LayerNorm, GELU activation, and dropout are incorporated into the encoder. The dropout probability is

$$p_{\text{drop}} = 0.10. \tag{280}$$

The final 32-dimensional output is treated as a latent representation rather than as an expected-return estimate.

B.5 Contrastive-Learning Configuration

Two stochastic graph views are generated for contrastive learning. The augmentation probabilities used in the experimental implementation are

Table 10: Graph augmentation probabilities.

View	Edge dropout	Feature masking
$G_t^{(1)}$	0.15	0.10
$G_t^{(2)}$	0.20	0.15

The projection head maps the 32-dimensional encoder representation through a 64-dimensional hidden layer and returns a 32-dimensional contrastive representation:

$$32 \to 64 \to 32. \tag{281}$$

GELU activation is used in the hidden layer, followed by ℓ_2 normalization of the projected representation.

The temperature parameter in the NT-Xent objective is

$$\tau = 0.2. \tag{282}$$

B.6 Optimization Parameters

The model parameters are optimized using Adam with

$$\eta = 10^{-3} \tag{283}$$

as the learning rate and

$$\omega = 10^{-5} \tag{284}$$

as the weight-decay coefficient.

The number of training epochs is

$$N_{\text{epoch}} = 10. \tag{285}$$

For the revised framework, the complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}}. \tag{286}$$

The value of λ_{CL} should be explicitly reported in the final experimental implementation. If the revised experiments use a value different from the original implementation, the revised value must replace the original value in the final manuscript.

B.7 Return Decoder Configuration

The return decoder is a supervised component introduced to map the latent representation into the return space:

$$h_{i,t} \longrightarrow \widehat{r}_{i,t+1}. \tag{287}$$

The decoder parameters are trained jointly with the Graph Transformer through the prediction loss and the contrastive-learning loss.

The return decoder should be regarded as a distinct module from the projection head. The projection head is used exclusively for contrastive representation learning, whereas the return decoder produces the quantity used as the expected-return input to the portfolio optimization problem.

B.8 Portfolio Optimization Parameters

The risk-aversion parameter in the Markowitz objective is set to

$$\gamma = 1. \tag{288}$$

The portfolio weights satisfy the full-investment condition

$$\mathbf{1}^{\top} \boldsymbol{w}_t = 1. \tag{289}$$

Any additional restrictions, such as long-only constraints or position limits, must be stated explicitly according to the actual implementation used to generate the final numerical results.

B.9 Rebalancing Frequencies

Four periodic rebalancing frequencies are considered:

$$21, \quad 63, \quad 126, \quad 252 \tag{290}$$

trading days.

These correspond approximately to monthly, quarterly, semiannual, and annual rebalancing, respectively.

A buy-and-hold configuration is also considered for comparison.

B.10 Benchmark Portfolios

The empirical evaluation includes the following benchmark configurations:

1. the revised Graph Transformer–Decoder–Markowitz framework;
2. conventional Markowitz optimization;
3. equal-weighted portfolio;
4. the S&P 500 benchmark.

For clarity, the notation used in the original implementation should be revised when reporting the new experiments. In particular, the label "MK(Embedding)" should not be used for the revised method because the embedding itself is not interpreted as an expected return. A label such as

$$\text{MK(GT-Decoder)} \tag{291}$$

more accurately describes the revised procedure.

B.11 Summary of Principal Parameters

Table 11 summarizes the principal configuration.

Table 11: Principal experimental and model parameters.

Parameter	Value
Outer training windows	$252, 504, 756$
Graph window	20 trading days
Number of edge features	5
Graph Transformer layers	3
Hidden dimension	64
Attention heads	4
Final embedding dimension	32
Dropout	0.10
Projection head	$32 \to 64 \to 32$
NT-Xent temperature	0.20
Learning rate	10^{-3}
Weight decay	10^{-5}
Training epochs	10
Risk aversion	$\gamma = 1$
Rebalancing frequencies	$21, 63, 126, 252$

B.12 Reproducibility Requirement for the Revised Model

The numerical results reported in the original implementation were obtained before introducing the supervised return decoder. Consequently, the numerical tables associated with the original "MK(Embedding)" formulation should not be presented as direct empirical results for the revised framework.

For the final version of the study, the complete experimental pipeline should be rerun after introducing the decoder:

$$G_t \to H_t \to \widehat{\boldsymbol{\mu}}_t \to \boldsymbol{w}_t^\star. \tag{292}$$

The same datasets, training windows, graph features, rebalancing frequencies, benchmark portfolios, and evaluation metrics should then be retained whenever possible. This permits the

revised method to be evaluated under a comparable experimental protocol while ensuring that the expected-return vector has the intended financial interpretation.

For reproducibility, the final implementation should additionally report the random seed, the exact train–validation procedure if validation data are used, the precise return-decoder architecture, the value of λ_{CL}, the portfolio constraints, and the treatment of missing observations and numerical covariance issues.

C Notation

For convenience, the principal notation used throughout the proposed framework is summarized below.

Table 12: Notation used in the proposed framework.

Symbol	Definition
n	Number of assets in the investment universe
t	Current portfolio formation date
T_w	Rolling graph and covariance estimation window
T_{train}	Outer model-training window
i, j	Asset indices
$r_{i,t}$	Realized return of asset i at time t
$\boldsymbol{r}_t$	Vector of realized asset returns at time t
$P_{i,t}$	Adjusted closing price of asset i at time t
X_t	Matrix of historical node features
$x_{i,t}$	Historical return feature vector of asset i
V	Set of graph nodes
E_t	Set of edges at time t
G_t	Financial graph at time t
A_t	Edge-attribute information of G_t
$a_{ij,t}$	Edge-attribute vector for (i, j)
q	Number of edge attributes
H_t	Matrix of Graph Transformer representations
$h_{i,t}$	Latent representation of asset i
d	Dimension of the latent asset representation
$\mathcal{N}_t(i)$	Neighborhood of asset i at time t
K	Number of attention heads
d_h	Dimension of an attention head
$f_\theta(\cdot)$	Graph Transformer encoder
θ	Parameters of the Graph Transformer encoder
$\mathcal{T}_v(\cdot)$	Graph augmentation transformation for view v
$G_t^{(v)}$	Augmented graph view v
$z_{i,t}^{(v)}$	Projection-head output for asset i
τ	Temperature parameter in the NT-Xent loss
$\mathcal{L}_{\text{CL}}$	Contrastive-learning loss
$g_\phi(\cdot)$	Supervised return decoder
ϕ	Parameters of the return decoder
$\widehat{r}_{i,t+1}$	Predicted next-period return of asset i
$\widehat{\boldsymbol{\mu}}_t$	Predicted expected-return vector
$\mathcal{L}_{\text{pred}}$	Return-prediction loss
λ_{CL}	Weight of the contrastive-learning loss
$\bar{\boldsymbol{r}}_t$	Historical sample mean-return vector
$\widehat{\boldsymbol{\Sigma}}_t$	Historical covariance matrix
γ	Markowitz risk-aversion parameter
$\boldsymbol{w}_t$	Portfolio-weight vector
$\boldsymbol{w}_t^\star$	Optimal portfolio-weight vector
$r_{p,t+1}$	Realized portfolio return
η	Learning rate
ω	Weight-decay coefficient
p_e	Edge-dropout probability
p_x	Node-feature masking probability

C.1 Distinction Between Core Quantities

Four quantities should be distinguished throughout the manuscript.

First, the Graph Transformer produces the latent representation

$$h_{i,t} \in \mathbb{R}^d. \tag{293}$$

This vector summarizes information learned from the financial graph and should not be interpreted as a return.

Second, the return decoder maps the latent representation into the return space:

$$h_{i,t} \longmapsto \widehat{r}_{i,t+1}. \tag{294}$$

Third, the predicted returns form the expected-return vector used by the portfolio optimizer:

$$\widehat{\boldsymbol{\mu}}_t = (\widehat{r}_{1,t+1}, \ldots, \widehat{r}_{n,t+1})^\top. \tag{295}$$

Fourth, the covariance matrix is estimated directly from historical realized returns:

$$\widehat{\boldsymbol{\Sigma}}_t = \frac{1}{T_w - 1} \sum_{s=t-T_w+1}^{t} (\boldsymbol{r}_s - \bar{\boldsymbol{r}}_t)(\boldsymbol{r}_s - \bar{\boldsymbol{r}}_t)^\top. \tag{296}$$

The resulting portfolio weights are then determined from

$$\boldsymbol{w}_t^\star = \arg\max_{\boldsymbol{w}} \left\{ \widehat{\boldsymbol{\mu}}_t^\top \boldsymbol{w} - \frac{\gamma}{2} \boldsymbol{w}^\top \widehat{\boldsymbol{\Sigma}}_t \boldsymbol{w} \right\}. \tag{297}$$

Thus, the principal distinction can be summarized by

$$\boxed{\text{Embedding} \neq \text{Expected Return} \neq \text{Covariance} \neq \text{Portfolio Weight.}} \tag{298}$$

This distinction is maintained throughout the revised formulation and is particularly important when interpreting the output of the Graph Transformer.

References

[1] H. Markowitz, "Portfolio Selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952. doi: https://doi.org/10.1111/j.1540-6261.1952.tb01525.x.

[2] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A Comprehensive Survey on Graph Neural Networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021. doi: https://doi.org/10.1109/TNNLS.2020.2978386.

[3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *International Conference on Learning Representations*, 2018. Available at: https://openreview.net/forum?id=rJXMpikCZ.

[4] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification," arXiv preprint arXiv:2009.03509, 2020. Available at: https://arxiv.org/abs/2009.03509.

[5] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," in *Proceedings of the 37th International Conference on Machine Learning*, PMLR, vol. 119, pp. 1597–1607, 2020. Available at: https://proceedings.mlr.press/v119/chen20j.html.

[6] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph Contrastive Learning with Augmentations," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 5812–5823, 2020. Available at: https://proceedings.neurips.cc/paper/2020/hash/3fe230348e9a12c13120749e3f9fa4cd-Abstract.html.

[7] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," arXiv preprint arXiv:1903.02428, 2019. Available at: https://arxiv.org/abs/1903.02428.

[8] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv preprint arXiv:1606.08415, 2016. Available at: https://arxiv.org/abs/1606.08415.

[9] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations*, 2015. Available at: https://openreview.net/forum?id=ty6L-6JtJWC.

[10] A. Chekhlov, S. Uryasev, and M. Zabarankin, "Drawdown Measure in Portfolio Optimization," *International Journal of Theoretical and Applied Finance*, vol. 8, no. 1, pp. 13–58, 2005. doi: https://doi.org/10.1142/S0219024905002767.

[11] P. Jorion, *Value at Risk: The New Benchmark for Managing Financial Risk*, 3rd ed., McGraw-Hill, 2007.

[12] R. T. Rockafellar and S. Uryasev, "Conditional Value-at-Risk for General Loss Distributions," *Journal of Banking & Finance*, vol. 26, no. 7, pp. 1443–1471, 2002. doi: https://doi.org/10.1016/S0378-4266(02)00271-6.

[13] M. C. Jensen, "The Performance of Mutual Funds in the Period 1945–1964," *The Journal of Finance*, vol. 23, no. 2, pp. 389–416, 1968. doi: https://doi.org/10.2307/2325404.

[14] W. F. Sharpe, "The Sharpe Ratio," *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994. doi: https://doi.org/10.3905/jpm.1994.409501.

[15] F. A. Sortino and L. N. Price, "Performance Measurement in a Downside Risk Framework," *The Journal of Investing*, vol. 3, no. 3, pp. 59–64, 1994. Available at: https://www.pm-research.com/content/iipmfin/3/3/59.

[16] C. Keating and W. F. Shadwick, "A Universal Performance Measure," *Journal of Performance Measurement*, vol. 6, no. 3, pp. 59–84, 2002. Available at: https://www.performance-measurement.org/KeatingShadwick2002a.pdf.

[17] O. C. Herfindahl, *Concentration in the U.S. Steel Industry*, Ph.D. dissertation, Columbia University, 1950.

[18] S&P Dow Jones Indices, *Dow Jones Averages Methodology*, S&P Dow Jones Indices. Available at: https://www.spglobal.com/spdji/en/documents/methodologies/methodology-dj-averages.pdf.

[19] R. Aroussi, *yfinance*, Python package. Available at: https://github.com/ranaroussi/yfinance.